



BASE24[®]

ITS Kernel Configuration Manual

© 2000 by ACI Worldwide Inc. All rights reserved.

All information contained in this document is confidential and proprietary to ACI Worldwide Inc. This material is a trade secret and its confidentiality is strictly maintained. Use of any copyright notice does not imply unrestricted or public access to these materials. No part of this document may be photocopied, electronically transferred, modified, or reproduced in any manner without the prior written consent of ACI Worldwide Inc.

ACI, BASE24, BASE24-atm, BASE24-pos, BASE24-telebanking, and NET24-XPNET are trademarks or registered trademarks of ACI Worldwide Inc., Transaction Systems Architects, Inc., or their subsidiaries.

Other companies' trademarks, service marks, or registered trademarks and service marks are trademarks, service marks, or registered trademarks and service marks of their respective companies.

Print Record

07/2000, Version 3 (Reissue)

03/97, Version 2 (Reissue)

07/95, Version 1 (New)

Contents

Preface	vii
Conventions Used in this Manual	xiii
Section 1	
Introduction	1-1
Integrated Transaction Services	1-2
Encapsulation	1-2
Object Architecture	1-3
Processes	1-3
Objects	1-4
Abstraction	1-4
Member Functions	1-5
Active and Passive Objects	1-5
Object Configuration	1-5
Object Classes	1-6
Kernel Objects	1-6
Core Objects	1-6
Integrated Endpoint Objects	1-7
Lexicons	1-7
Instances	1-8
Attributes	1-8
Server Classes	1-8
Extended Memory Tables	1-9
Integration	1-11
Number of ITS Processes	1-11
Server Classes	1-11
Application Partitioning	1-12
Process Identification Number (PIN)	1-13

Introduction *continued*

Memory Pool Usage	1-14
Event Suppression.....	1-15
ITS Configuration Files	1-16
Kernel Configuration Files	1-16
Other Supporting Files	1-16
ITS Configuration Procedures	1-17
Event Messages.....	1-18
Event Message Documentation.....	1-19

Section 2

ITS Configuration Files.....	2-1
Kernel Configuration Files	2-3
Default Data	2-3
Customer-Configured Data.....	2-3
Kernel Configuration File Requirements	2-4
Object-Oriented Processes	2-4
Procedural Processes.....	2-6
Logical Network Configuration File.....	2-7
Pathway Configuration File	2-8
Network Environment File	2-9
Source Class	2-9
Process Names	2-9
Queue Management	2-9
Network Overload Management Percentage	2-10
Network Overload Percentage Threshold.....	2-10
Network Overload Management Actions	2-11
Library Usage	2-12
Example Definitions	2-12
ITS Process Example	2-12
ITS Station Example.....	2-12
Measure Configuration	2-14

Section 3

ITS Configuration Procedures	3-1
Process Binding Requirements	3-2
Process Control Configuration Procedures	3-3
Prerequisites	3-3
File Relationships and Dependencies	3-3
Procedures	3-3
Modifying Default Data	3-15
Adding New Assigns	3-15
Adding New Extended Memory Tables	3-16
Adding New Objects	3-17

Section 4

Extended Memory Table Build Utility	4-1
Overview	4-2
Allocating a Newly Built or Rebuilt Extended Memory Table	4-4
Object-Oriented Processes	4-4
Procedural Processes	4-4
Source Files and Tables	4-6
Event Suppression File Extended Memory Table	4-6
Source Class Map File Extended Memory Table	4-6
Source Symbolic Map Extended Memory Table	4-6
LCONF Assigns and Params	4-7
Startup	4-8
Extended Memory Table Build Report	4-10
Cover Page	4-10
Hash Table Summary Page	4-11
Trailer Page	4-14

Section 5

Event Suppression	5-1
Overview	5-2
Event Object	5-2
Log Permission Object	5-2

Overview *continued*

Event Suppression Files	5-3
Event Suppression File	5-3
Event Suppression File Extended Memory Table	5-3
Log Permission Process Partition File	5-3
Event Suppression Levels	5-4
Logged Events	5-4
Not Logged Events	5-4
Process-Controlled Events	5-5
Log Permission Process-Controlled Events	5-5
Event Suppression Methods	5-6
Suppression by Number of Events	5-6
Suppression by Time Interval	5-6
ITS Process Configuration	5-8
Object Requirements	5-8
LCONF Requirements	5-8
Event Suppression Configuration	5-9
General Configuration Requirements	5-9
Suppressing an Event at the Network Level	5-10
Suppressing an Event at the Process Level	5-10
Never Logging an Event	5-10
Always Logging an Event	5-11
Performance Considerations	5-12
Transaction Path Impacts	5-12
Arbitration of Events by Logical Network	5-12
Arbitration of Events by Process	5-13
Not Logging	5-13
Logging	5-13

Glossary	Glossary-1
-----------------------	-------------------

Index	Index-1
--------------------	----------------

Preface

This manual describes the configuration files and procedures required to configure the integrated transaction services (ITS) environment. These procedures are intricate and critical to system performance. Therefore, it is strongly suggested that these procedures be performed only by the system manager.

All ITS components (kernel, integrated endpoint, and core) are configured using a common set of procedures and configuration files. The same configuration procedures are used whether your ITS environment resides in a single resource node in a single logical network, or in many resource nodes stretched across geographically distributed Tandem nodes.

Audience

The ***BASE24 ITS Kernel Configuration Manual*** is intended for users responsible for configuring the integrated transaction services (ITS) environment. The manual serves as a reference guide for implementing an ITS environment configuration.

This manual describes the concepts, files, and procedures involved in configuring the ITS environment. These procedures are intricate and critical to system performance. Therefore, it is strongly suggested that these procedures be performed only by the system manager.

Prerequisites

The ***BASE24 ITS Kernel Files Maintenance Manual*** should be read in conjunction with this manual. Many of the procedures presented in the ***BASE24 ITS Kernel Configuration Manual*** use the file maintenance screens presented in the ***BASE24 ITS Kernel Files Maintenance Manual***.

Readers may also find the ***BASE24 ITS Kernel Objects Reference Manual*** and ***BASE24 IEP Objects Reference Manual*** helpful when configuring objects.

Additional Documentation

The BASE24 documentation set is arranged so that each manual presents a topic or group of related topics in detail. When one manual presents a topic that has already been covered in detail in another manual, the topic is summarized and the reader is directed to the other manual for additional information. Information has been arranged in this manner to be more efficient for readers who do not need the additional detail and at the same time provide the source for readers who require the additional information. This manual contains references to the following BASE24 publications:

- The ***BASE24 Text Command Reference Manual*** provides detailed information on commands used to warmboot processes that run under control of the XPNET process.
- The ***BASE24 CRT Access Manual*** provides brief descriptions for each of the ITS configuration and control screens.
- The ***BASE24 Logical Network Configuration File Manual*** provides detailed descriptions for each of the Logical Network Configuration File (LCONF) assigns and params used by kernel objects.
- The ***BASE24 ITS Kernel Files Maintenance Manual*** describes the ITS configuration and control screens. These screens are used when configuring or controlling any object-oriented ITS process.
- The ***BASE24 Kernel Objects Reference Manual*** describes each of the kernel objects from which an object-oriented ITS process is composed and shows how each object is used in transaction processing.
- The ***BASE24 IEP Configuration Manual*** describes how Integrated Endpoint processes are configured for message routing purposes.
- The ***BASE24 IEP Objects Reference Manual*** describes each of the integrated endpoint objects from which an Integrated Endpoint process is composed and shows how each object is used in transaction processing.

In addition to the above BASE24 manuals, this manual contains references to the following NET24 publications:

- The ***NET24-XPNET Application Programming Guide*** describes the fields used in the message header to support network overload management.

- The *NET24-XPNET Event Management Manual* contains detailed information on modifying template source files and the ViewPoint EVENTCX file. The EVENTCX file is an optional file that contains event detail information about event messages generated by an application subsystem.
- The *NET24-XPNET Network Control Command Reference Manual* describes the commands and attributes used to configure processes and stations to the XPNET system.
- The *NET24-XPNET Network Control Operations Guide* provides detailed information on attributes used to detect network overload situations.
- The *NET24 Performance Monitoring Manual* provides detailed information on performance monitoring facilities and procedures.

This manual also contains references to the following Tandem publications:

- The *Binder Manual* describes how you can use the Tandem Binder product to determine the objects that make up an executable process.
- The *Guardian Programmer's Guide* contains detailed information on memory usage and application frames.
- The *NonStop TS/MP System Management Manual* describes commands used to reinitialize Pathway servers that use extended memory tables in the ITS environment.
- The *TACL Reference Manual* contains information on how different HIGHPIN options affect whether a new process runs at a high or low PIN.
- The *ViewPoint Manual* contains information on the EVENTCX file.

Software

This manual documents standard processing for Integrated Transaction Services release 1.3.

Customer-specific code modifications (that is, CSMs) may result in processing that differs from the material presented in this manual. The customer is responsible for identifying and noting these changes.

Manual Summary

The following is a summary of the contents of this manual.

“Conventions Used in this Manual” follows this preface and describes notation and documentation conventions necessary to understand the information in the manual.

Section 1, “Introduction,” introduces the ITS environment and its configurable entities. It includes a discussion of the following topics: object architecture, configuration files, configuration procedures, integration of the ITS environment, and event suppression.

Section 2, “ITS Configuration Files,” introduces you to each of the files involved in configuring an ITS processing system.

Section 3, “ITS Configuration Procedures,” provides step-by-step procedures for configuring the ITS environment.

Section 4, “Extended Memory Table Build Utility,” provides detailed instructions for using the Extended Memory Table Build utility to build shared extended memory tables from disk files or tables. Extended memory tables must be built when configuring the ITS processing environment and rebuilt whenever data contained in a source disk file or table is changed.

Section 5, “Event Suppression,” discusses event suppression and how it is used in the ITS environment. It also describes how event suppression is configured to the ITS environment.

A glossary that contains definitions of terms used in this manual follows appendix A.

Publication Identification

Two entries appearing at the bottom of each page uniquely identify this BASE24 publication. The publication number (for example, OK-AE000-01 for the ***BASE24 ITS Kernel Configuration Manual***) appears on every page to assist readers in applying updates to the appropriate manual. The publication date (for example, 07/2000 for July, 2000) identifies the last time material on a particular page has been revised. This date can be used to verify that the reader has the most current information and is particularly helpful when dealing with HELP24 to answer operating questions.

The print record, located on the page immediately following the title page, is also helpful when verifying that a manual contains the most current information. An entry is added to the print record each time a BASE24 publication is modified in any way. Each print record entry includes the date of the change, a unique version number, and the type of change (new, update, reissue, or preliminary). Version 1 is assigned to a *new* manual. Later versions can be updates or reissues, depending on the changes being made. An *update* typically involves changes on a small number of pages throughout the manual. A *reissue* occurs when all pages of the manual are replaced because changes affect a large number of pages. A manual is identified as *preliminary* when it is released to customers on a limited basis, but it has not yet reached final form.

ACI Worldwide Inc.

Conventions Used in this Manual

This section explains how syntax notation is documented in this manual. It also explains a change in Nucleus process terminology.

Syntax Notation

Syntax descriptions in this manual use several symbols and conventions to denote how commands must be entered. These conventions are described in the table below.

Notation	Meaning
UPPERCASE LETTERS	Indicate key words and literals that you must enter exactly as shown.
<i>lowercase italics</i>	Identify variables that you must provide.
Brackets []	Enclose optional syntax items that you can enter in the command.
Braces { }	Enclose a group of items from which you are required to choose one item. The items are separated within the braces by a vertical bar ().
Vertical bar	Separates alternatives in a list of items.
Ellipsis ...	Indicates the immediately preceding syntactical item can occur one or more times.
Punctuation	Must be entered as shown. This includes parentheses, commas, semicolons, and other symbols not previously described.

Notation	Meaning
Spaces	Are required as delimiters wherever it would be otherwise impossible to discriminate between adjacent words or symbols. You can insert additional spaces between any adjacent words or symbols, but not within a word or multiple character symbol.
Indented lines	Indicates a continuation of the preceding line of syntax. If the syntax of a command is too long to fit on a single line, each continuation line is indented.

Nucleus Process Terminology Change

There is one process that is the hub of an XPNET node. It manages message queues; controls all lines, stations, and processes in the system; and serves as an interprocess interface. Throughout this manual, this process is referred to as the XPNET process.

In other manuals, you may see references to the Nucleus process as performing these same functions. These two terms (Nucleus process and XPNET process) should be viewed as synonymous. The Nucleus process and the XPNET process perform the same functions and processing. As manuals are reissued, references to the Nucleus process will be replaced with references to the XPNET process.

Section 1

Introduction

To allow applications to be compatible with the integrated transaction services (ITS) architecture, ACI provides a standard interface and set of procedures used in developing object-oriented applications. Consequently, all ITS components (that is, kernel, endpoint, and core) are configured using a common set of procedures and configuration files. The same configuration procedures are used whether your ITS environment resides in a single XPNET node in a single logical network, or in many XPNET nodes across geographically disbursed Tandem nodes.

This section introduces you to the concepts, files, and procedures involved in configuring the ITS environment.

Integrated Transaction Services

Integrated transaction services (ITS) is a conceptual framework for designing and developing products utilizing an object-oriented design philosophy. Object-oriented design is a method of design that provides small, easy to understand, reusable components, and a consistent interface. Software programs produced using the object-oriented methodology are typically easier to understand and maintain than programs produced using a traditional procedural function-oriented methodology, saving both time and money.

The BASE24 ITS environment interfaces with the XPNET system for queue and data management, auditing, and controlling interprocess communications. The XPNET system also allows the BASE24 ITS environment to interface with Tandem subsystems such as the Event Management Service (EMS), Measure, and Pathway. The XPNET system is controlled by the XPNET process.

The ITS environment can also be integrated with existing BASE24 procedural applications, allowing for parts of an application to be converted to the ITS architecture, while other parts remain in a procedural environment. This allows for an evolutionary approach to converting a procedural application into an ITS-based, object-oriented application.

The ACI implementation of object-oriented programming is based upon the concept of encapsulation, which is described below.

Encapsulation

Encapsulation refers to the presence of objects—self-contained modules of code defining program data and the operations that can be performed upon that data. An object-oriented program is built entirely from these self-contained objects, which communicate by sending messages to one another.

In a traditional procedural, function-oriented program, data and operations are separated. For example, you might have an array containing account data and a separate printing function. To print the account data, the print function must read and interpret the array, process the data, and send it to the spooler.

In an object-oriented environment, you would instead have a single, self-contained account object. This object would contain not only the account data, but also would contain the program logic necessary to debit the account, credit the account, and print account details. To print account data, an object-oriented program would send a print message to the account object, which would do the rest.

Object Architecture

The ITS software environment is flexible and modular in design. Because of this flexibility and modularity, you can configure an ITS system to support multiple applications at both small and large installations simply by increasing or decreasing the number of system entities. This subsection introduces you to the various system entities that compose an ITS configuration and how these entities can be manipulated in the configuration to support the needs of your installation.

Every ITS configuration is composed of processes, objects, lexicons, instances, attributes, assigns, params, defines, server classes, files or tables, and extended memory tables. You can think of each of these items as a system entity. Some of these entities are configurable, while others are not. Determining your unique ITS system needs, and then relating the system entities to each other to best meet those needs, is the process of system configuration. Each of these system entities and how they relate to each other is explained below and on the following pages.

Processes

In the ITS environment, all object-oriented processes are defined to the XPNET system and to the ITS environment in the Process File (PRO). The ITS environment also uses and can interface with procedural processes running under control of the XPNET process or the Pathway process. Processes running under control of the Pathway process are configured in the Pathway Configuration File (PATHCONF).

Object-oriented processes are actually composed from a set of objects bound together into one executable program. You can think of objects as the building blocks that make up a process, and you can think of the process as an envelope containing the processing environment in which the objects run. Objects typically provide small, easily manageable units of code that are reusable and can accommodate high transaction volumes. Objects are related to the processes in which they run using the Process/Object Relationship File (PORF).

A process type is associated with each process for the proper distribution of process control commands. Process types are configured in the PRO and described in the Process Type File (PTF). Processes that can reread individual assigns and reallocate extended memory tables on command are related to those assigns and extended memory tables by process type using the Assign/Process Type Relationship File (APRF) and the EMT/Process Type Relationship File (EPRF), respectively.

The ITS environment does not use object-oriented processes exclusively. Procedural processes are still used for some functions. For example, the Integrated Transaction Services Control Server process is a procedural process. This process runs under the Pathway process and is responsible for delivering operator-initiated commands to object-oriented processes.

Note: Any procedural processes that use the Integrated Server Control File (ISCF) to receive process control commands must also be configured in the PTF.

Objects

An object is a combination of privately held data and functions that act upon that data. An object is called by sending it a message. The object interprets the message and performs the appropriate action. The functions performed by objects are typically referred to as member functions. The fundamental difference between procedural-based programming and object-oriented programming is the coupling of data with functions. Objects are bound into each process in which they run.

Each object runs in its own space in memory called a frame. A frame contains local variables, saved registers, and parameters to called procedures. For more information on frames, refer to the *Guardian Programmer's Guide*.

Abstraction

An object is an abstraction of some concrete element (for example, customers, accounts, and institutions) in the problem domain (for example, remote banking). Objects represent real things that exist in the problem domain. You can think of an object as a package of code that performs specific functions. Objects are tightly encapsulated in that much of the underlying data structure and means of accessing that data structure are hidden from the user of the object because of a well-defined interface. In their relationships with each other, objects can be considered to be loosely coupled modules. One object cannot access another object's data unless it uses the appropriate member functions. Objects are like black boxes in that the user of the object does not care how the object operates or what data structures it uses. The user of the object only needs to know how to use the object to accomplish a specific task.

Member Functions

Objects cooperate with one another using messages that trigger member functions. Groups of inter-object member functions are called lexicons. Lexicon messages between objects always move from a client to an actor. The sender of the object is considered to be the client object while the recipient of the message is considered to be the actor object. The actor object executes the member function and returns the response data, if there is any. While executing the member function, the actor object can call upon other objects to perform additional member functions. Every object in the ITS environment can function as both a client and an actor. Each message causes the actor object to perform one member function.

All member functions fall into one of two categories—common-supplied or public. Common-supplied member functions are those functions supported by an object that are common to most objects. An example of a common-supplied member function is the LCONF Warmboot request. Public member functions include any functions performed specifically on behalf of a particular object. An example of a public member function is the Memory Get request.

Active and Passive Objects

An object can also be either active or passive. Active objects contain a thread of control for the ITS system or an application. Passive objects do not have a thread of control for the ITS system or an application. For example, passive objects supply information to other objects, but do not directly control a transaction.

An active object remains on the stack during the entire transaction path, passing control off to other objects to perform their functions using the appropriate lexicon messages. A passive object receives control, performs the desired processing, responds, and is removed from the stack.

Object Configuration

Objects are configured to the ITS environment in the Object File (OBJ) and related to the processes in which they run in the PORF. Objects that can reread individual assigns and reallocate extended memory tables on command are related to those assigns and extended memory tables using the Assign/Object Relationship File (AORF) and the EMT/Object Relationship File (EORF), respectively.

Object Classes

An object class is a set of objects sharing a common structure and behavior. A class acts as a template for an object. Three classes of objects are allowed in an ITS environment, each of which is described below.

Only the kernel and integrated endpoint classes of objects are provided as part of the BASE24 ITS products. The core class of objects is provided as needed for specific applications running in the ITS environment, such as the BASE24-telebanking product.

Object classes are not configured in the ITS environment.

Kernel Objects

Some objects provide functionality required by most processes in an ITS system. These objects are called kernel objects. Some kernel objects must be bound into every ITS process, while others are bound into a single process. Kernel objects provide services essential to object-oriented processes running in the ITS environment.

An example of a kernel object is the Application Network Services object. This object contains the main procedure for an object-oriented process. Its primary function is to set up an environment for all objects in the process to use and provide the mechanism for inter-object communication within the process. Therefore, the Application Network Services object is bound into every ITS process.

An example of a kernel object used by only one type of process is the Log Permission object. This object resides in Log Permission processes only, and its sole purpose is to determine whether a particular event message is to be logged or suppressed at the network level.

Core Objects

Some objects provide functionality that is essential to the successful authorization processing of most applications. These objects provide business services and are referred to as core objects. Core objects are used to build applications. Many of these objects can be reused from one application to another.

An example of a core object is the Account Access object used by the BASE24-telebanking application. This object provides access to customer accounts.

Integrated Endpoint Objects

Objects that provide message-based routing related functions are called integrated endpoint objects. These objects provide functionality specific to the processing required for Integrated Endpoint processes. These processes provide an interface to a particular transaction endpoint, such as a remote banking endpoint device.

An example of an integrated endpoint object is the Context Manager object. This object is responsible for storing and retrieving message context over the course of a transaction.

Lexicons

A lexicon is a convention used to group member function messages that have either a common client, actor, or purpose. Lexicons are groups of messages used to provide communication between objects. Lexicons have nothing specifically to do with object-oriented design, they are simply provided as a standard means for organizing DDL structures in a logical manner. For objects to be able to communicate with each other, they must share at least one of the same lexicons. Messages indicate what member function needs to be performed by the actor object and contain any data required to perform the function. A member function must be specified in every lexicon message. Some member functions, however, do not require any data.

Lexicon messages can be nested within one another. For example, a transaction response lexicon message can be nested within a Message Out member function message when it is sent from the local process to an outside agency. The only limit as to the number of messages that can be nested within one another is the physical size of the message buffer used.

A library of member function lexicons used by kernel objects is provided in the ***BASE24 ITS Kernel Objects Reference Manual***. Lexicons are not configured in the ITS environment.

Instances

An instance is the occurrence of a specific system or application entity. In the ITS kernel environment, for example, objects register with the Application Network Services object when they initialize. The Application Network Services object adds an entry to an internal table for each object that registers within the process. Each entry in this table can then be said to represent the instance of a particular object within a process. Instances are not configured in the ITS environment.

Attributes

Attributes allow you to assign specific processing variables to ITS processes and objects. Object-oriented process and object attributes can be defined in the XPNET system, in Logical Network Configuration File (LCONF) assigns and params, in SQL defines, and in various (kernel, IEP, and core) configuration files and tables specified by an LCONF assign or SQL define. Attributes control both the runtime processing environment in which processes and objects execute and how transaction processing is performed.

Note: Runtime attributes for procedural processes running in the ITS environment under control of the Pathway process are configured in the Pathway Configuration File (PATHCONF).

Server Classes

A server class defines a pool of identical servers whose number can vary with changes in workload, thereby providing scalability in performance. By assigning an entire group of identical servers to a server class, a client (sending object) does not have to open a specific server of the class; rather, it opens the class and is assigned a link to an available server within that class. To allow for this, all server classes must remain context-free. The context-management function of the Integrated Endpoint process allows all authorization servers running in the ITS environment to remain context-free.

Server classes can contain internal servers, external servers, or a mixture of both. Internal servers are objects residing within the Application Network Services object's home process that have registered at least once as a server. These servers are within the processing domain of the home process and, thus, are considered to be internal to the process. External servers are objects or processes that reside outside of the home process.

Processes running in the ITS environment are assigned a server class type for the proper distribution of process control commands or transaction messages. Each server class type has a unique Integrated Server Control File (ISCF) to which messages intended for that server class are written. Servers within a class continually monitor their ISCF for incoming messages. Server class types are configured in the Server Class Type File (SCT) and related to processes in the Server Class Type/Process Relationship File (SPRF).

Note: Any procedural processes that use the ISCF for command or message delivery must also be configured in the SCT and SPRF.

Extended Memory Tables

For performance reasons, components of the ITS environment access critical Enscribe file or SQL table data from extended memory tables held in processor memory rather than from files or tables on disk. This saves time in the execution of time-critical operations. Some extended memory tables are shared by more than one object while others are used by a single object only. The Extended Memory Table Build utility provided with the ITS environment is used to build shared extended memory tables from disk files or tables. Extended memory tables that are not shared are built by the individual objects that use them. Once an extended memory table has been built or rebuilt, it must be allocated or reallocated before it can be used in processing.

Individual extended memory tables that can be allocated using the EMT Warmboot process control command or that can be built by the Extended Memory Table Build utility are configured in the Extended Memory Table File (EMT). They are related to the objects that can allocate them using the EMT WARMBOOT command in the EMT/Process Type Relationship File (EPRF) and the EMT/Object Relationship File (EORF).

The EMT WARMBOOT command causes an object to validate the name of the extended memory table, allocate the new extended memory table, deallocate the old extended memory table, and replace the extended memory table segment ID in the frame. If more complicated processing is required for an extended memory table (for example, building a secondary frame), then you cannot use the EMT WARMBOOT command to allocate or reallocate the extended memory table. In this case, you can allocate or reallocate the extended memory table by reinitializing the object that uses the table (using the LCONF WARMBOOT command) or by restarting the process in which the object is contained.

Once built, extended memory tables can be used by both ITS-based and procedural-based processes, although the EMT WARMBOOT and LCONF WARMBOOT commands can only be used with ITS processes.

Integration

The following paragraphs discuss issues relating to the integration of ITS software into different processing environments.

ITS software is flexible in that it can be integrated into existing procedural-based applications such as BASE24-atm or BASE24-pos, used exclusively with ITS-based applications such as BASE24-telebanking, or be combined with both in a hybrid procedural/ITS application environment.

ITS software interfaces with existing procedural-based applications through the use of an Integrated Endpoint process for message routing. Integrated Endpoint processes can support both message-based routing (routing based on the message source and message class) and symbolic destination routing (routing based on the message content and the logic built directly into the Integrated Endpoint process). Message-based routing uses a configuration file to determine message destinations while symbolic destination routing does not. Message-based routing can determine multiple destinations for a message while symbolic destination routing can only determine a single destination for a message. For any given message, message-based routing, symbolic destination routing, or both methods of routing can be used. In the latter case, the message is routed to a single symbolic destination determined through symbolic destination routing as well as to any additional destinations determined through message-based routing. For detailed information on the integration issues for an Integrated Endpoint process, refer to the ***BASE24 IEP Configuration Manual***.

Number of ITS Processes

The number of ITS-based processes required for your system is dependent upon many factors, including your transaction throughput requirements, contingency requirements if one or more ITS-based processes is unavailable, as well as other business requirements.

Server Classes

Server classes allow you to scale the number of servers to best meet your performance requirements. As the transaction load increases or decreases, you can add servers to a class or delete servers from a class, respectively.

Application Partitioning

Transactions can be routed to different ITS-based or procedural-based application processes or stations using message-based routing, symbolic destination routing, or both methods of routing. These routing methods are implemented through the use of Integrated Endpoint processes and allow you to effectively partition the workload among applications.

Process Identification Number (PIN)

XPNET release 2.1 or greater supports the extended features of the Tandem D-series operating system that enable a process to run with a high process identification number (PIN). A PIN is a unique, system-assigned identifier with a numeric value that identifies processes running in a CPU. The Tandem D-series operating system allows up to 65,534 processes to run simultaneously in a single CPU. High PIN is defined as a process identification number in the range of 256 to 65,534. Low PIN refers to a process identification number in the range of 0 to 254. Both the XPNET process and all ITS processes controlled by the XPNET process can be started as either low or high PIN processes.

All processes started by ITS processes (for example, a report process started by the End-of-Period process) are started as low PIN processes only.

The Extended Memory Table Build utility, is started as a low PIN process when executed from the Configuration Control screen (CCS). However, this utility can be started as a high or low PIN process when executed from a TACL prompt.

A process runs with a high PIN when the executable object file contains the HIGHPIN file attribute, or if you start the process from TACL with the HIGHPIN ON attribute.

Note: High PIN functionality is not available if your ITS environment runs under XPNET release 2.0.

Memory Pool Usage

XPNET release 2.1 or greater supports the extended features of the Tandem D-series operating system that enable a process to use a large-memory model as well as a small-memory model. ITS processes running under control of the XPNET process can use either memory model. Using the small-memory model, ITS objects can allocate space in the upper 64K bytes (32K words) of the data stack and use message buffers of 4096 bytes. Using the large-memory model, ITS objects can allocate space within a flat segment. Flat segments allow memory space to be larger than 65,536 bytes, use message buffers of 32,000 bytes, and require a D30 or later operating system running on a Tandem NonStop Series/RISC machine.

A large-memory model toggle set when the Application Network Services object is compiled, determines whether the small-memory model or large-memory model is used. If the toggle is set, the large-memory model is used. If the toggle is not set, the small-memory model is used.

Note: The large-memory model is not available if your ITS environment runs under XPNET release 2.0 or if your Tandem system is not RISC-based.

Event Suppression

Event suppression is a means of reducing the number of times a particular log message is generated. Events can be suppressed for a specified time interval or for a specified number of occurrences, and can be suppressed at the process or logical network level. Event suppression is needed when multiple processes log the same event message or when the same event message is continually generated by the same process.

Considerations for configuring event message suppression are provided in section 5.

ITS Configuration Files

The following paragraphs introduce the files involved in configuring the ITS environment. This does not include the files and tables that must be configured on behalf of the Integrated Endpoint process or ITS applications. For Integrated Endpoint process configuration information, refer to the ***BASE24 IEP Configuration Manual***. For ITS application-specific configuration information, refer to the product-specific transaction processing manual.

Each of these files is described in section 2. Step-by-step procedures for configuring them are provided in section 3.

Kernel Configuration Files

Kernel configuration files are used to define each of the following entities as well as the interrelationships that exist among these entities: processes, process types, objects, assigns, extended memory tables, and server class types. The interrelationship files are used for process control and process communication purposes. Kernel files are also used to configure event suppression.

The screens and fields used to maintain these files are described in detail in the ***BASE24 ITS Kernel Files Maintenance Manual***. A brief introduction to each of these screens is provided in the ***BASE24 CRT Access Manual***.

Other Supporting Files

In addition to the kernel files, configuration information for ITS-based processes is included in the following files. These files are not specific to the ITS architecture.

- Logical Network Configuration File (LCONF)
- Network Environment File (NEF)
- Pathway Configuration File (PATHCONF)
- Measure Configuration File

ITS Configuration Procedures

To ensure a valid configuration, the ITS environment should be configured in a systematic, organized way. Items in some files must be configured before they can be used or related in other files.

Section 2 lists each of the LCONF assigns and params that must be configured specifically on behalf of kernel objects and files in one of the configuration steps.

Section 3 provides detailed step-by-step procedures for configuring the ITS environment. It includes a discussion of process binding, which must be done prior to configuration when the software is installed, and steps for configuring the ITS environment. This section also includes directions for modifying any of the default data provided by ACI with the configuration files.

Section 4 provides detailed instructions for using the Extended Memory Table Build utility. This utility is used to construct extended memory tables from Enscribe files or SQL tables on disk.

Section 5 provides detailed instructions for implementing event suppression.

Event Messages

ACI supplies template source files that provide information about each Event Management Service (EMS) event generated by ITS kernel objects and processes. The following template source files are provided with the ITS kernel environment.

Template File	Subvolume	Contains Events For
ANSPS	OK13ANS	Application Network Services object
CLASPS	OK13CLAS	Class object
KEMTBPS	OK13EMTB	Extended Memory Table Build utility for kernel objects
EVTPS	OK13EVT	Event object
KISMTPS	OK13ISMT	Shared integrated server (IS) module type objects
KLBPROPS	OK13ISMT	Kernel library procedure names
KLBSTAPS	OK13ISMT	Library status detail code
LOGPPS	OK13LOGP	Log Permission object
KOBSTAPS	OK13ISMT	Object status code
PCTLPS	OK13PCTL	Process Control object
TRCPS	OK13TRC	Trace object
XPNPS	OK13XPN	XPNET Interface object

As needed, you can modify these files to change the text of event messages generated by kernel objects and processes. The ***NET24-XPNET Event Management Manual*** contains detailed information on modifying template source files and the ViewPoint EVENTCX file. The EVENTCX file is an optional file that contains event detail information about event messages generated by an application subsystem. For more information on the EVENTCX file, see the Tandem ***ViewPoint Manual***.

Note: The KISMTPS, KLBPROPS, KLBSTAPS, and KOBSTAPS files are used by every object and are not compiled separately. Any changes made to these files require that you recompile all other template files.

Event Message Documentation

Event message documentation for the Integrated Transaction Services product is contained in files located in the OK13LOGM subvolume. Each file contains event messages for a specific BASE24 module or object, including the ITS kernel modules and objects. The files are organized according to product module ID (PMID). All kernel product module IDs begin with ISKN. One PMID exists for each BASE24 module or object. The file names for ITS kernel modules and objects are as follows:

File Name	Product Module Name
ISKNANS	Application Network Services object
ISKNCLAS	Class object
ISKNEMTB	Extended Memory Table Build utility
ISKNEVT	Event object
ISKNLOGP	Log Permission object
ISKNPCTL	Process Control object
ISKNTRC	Trace object
ISKNXPXN	XPNET Interface object

IEP event message documentation files are identified with a product module ID beginning with ISIE. Core event message documentation files are identified with a product module ID beginning with ISCO.

Note: See the README file in the BA53LOGM subvolume for instructions on printing event message documentation.

ACI Worldwide Inc.

Section 2

ITS Configuration Files

A number of files are used to configure the integrated transaction services (ITS) environment. These files are listed below. The disk file name matches the file name acronym used in the documentation.

- Assign File (ASGN)
- Assign/Object Relationship File (AORF)
- Assign/Process Type Relationship File (APRF)
- Extended Memory Table File (EMT)
- Extended Memory Table/Object Relationship File (EORF)
- Extended Memory Table/Process Relationship File (EPRF)
- Event Suppression File (ESF)
- Log Permission Process Partition File (LPF)
- Object File (OBJ)
- Process File (PRO)
- Process/Object Relationship File (PORF)
- Process Type File (PTF)
- Server Class/Process Relationship File (SPRF)
- Server Class Type File (SCT)
- Source Class Map File (SCM)

Assigns and params used by ITS components must be configured to the Logical Network Configuration File (LCONF).

All ITS processes run under control of the XPNET process and must be configured in the Network Environment File (NEF).

Some procedural processes used in the ITS environment, such as the ITS Control File Server process, run under control of the Pathway process and must be configured in the Pathway Configuration File (PATHCONF).

If transaction processing rates are to be tracked on behalf of ITS processes, the Measure configuration file must be configured accordingly.

Kernel Configuration Files

This subsection discusses the kernel file configuration requirements for object-oriented and procedural processes running in the ITS environment.

Default Data

Some of the configuration data required for ITS processes may be provided on a default basis. If so, you are not required to enter that data. Any default data provided for integrated transaction services processes is listed for each of the applicable kernel configuration files in the ***BASE24 ITS Kernel Files Maintenance Manual***. This manual lists the preconfigured default data provided for kernel objects and ACI-developed applications and includes the preconfigured data for kernel, integrated endpoint, and core objects.

Customer-Configured Data

Data in some of the ITS files must be configured during or after installation to meet your specific processing requirements. The following ITS files require some configuration after your ITS system is installed.

- Event Suppression File (ESF)
- Log Permission Process Partition File (LPF)
- Process/Object Relationship File (PORF)
- Process File (PRO)
- Server Class Type File (SCT)
- Server Class/Process Relationship File (SPRF)
- Source Class Map File (SCM)

Kernel Configuration File Requirements

The following paragraphs discuss the kernel file configuration requirements for ITS processes as well as procedural processes that use the process control mechanism of the ITS environment.

Object-Oriented Processes

The following kernel files contain the kernel configuration for ITS processes.

Assign File (ASGN)	Contains one record for each file assign used by an ITS object that can be reread separately. If an assign cannot be reread separately using the ASSIGN WARMBOOT command, it does not need to be configured in this file. Such assigns are reread only when you issue an LCONF WARMBOOT command.
Assign/Object Relationship File (AORF)	Establishes relationships between assigns and objects, with one record for each relationship between an assign configured in the ASGN and an object that can reread the assign using the ASSIGN WARMBOOT command.
Assign/Process Type Relationship File (APRF)	Establishes relationships between assigns and process types, with one record for each relationship between an assign configured in the ASGN and a process type of a process containing an object that can reread the assign using the ASSIGN WARMBOOT command.
Extended Memory Table File (EMT)	Contains one record for each extended memory table built by the Extended Memory Table Build utility or that can be reallocated using the EMT WARMBOOT command. If an extended memory table is not built by the Extended Memory Table Build utility or cannot be reallocated using the EMT WARMBOOT command, it does not need to be configured in this file.
Extended Memory Table/Object Relationship File (EORF)	Establishes relationships between extended memory tables and objects, with one record for each relationship between an extended memory table and an object that can reallocate the table using the EMT WARMBOOT command. If an extended memory table cannot be reallocated using the EMT WARMBOOT command, it does not need to be configured in this file.

Extended Memory Table/Process Type Relationship File (EPRF)	Establishes relationships between extended memory tables and process types. One record is required for each relationship between an extended memory table configured in the EMT and a process type. The process type refers to each type of process containing an object that can reallocate the extended memory table using the EMT WARMBOOT command.
Event Suppression File (ESF)	Defines events that are to be suppressed, with one record for each event that is to be suppressed in some way. This file is used to build the Event Suppression File extended memory table, which is used by the Event and Log Permission objects.
Log Permission Process Partition File (LPF)	Contains a record for each range of subsystem IDs (SSIDs) used in the ITS system for which events are to be suppressed at the network level. The file maps ranges of SSIDs to specific Log Permission processes responsible for arbitration when determining whether an event is to be logged. This file is used by the Log Permission object.
Object File (OBJ)	Contains one record for each object used in the ITS environment. This file defines the process control capabilities for each object.
Process File (PRO)	Contains one record for each ITS process that can receive process control commands.
Process/Object Relationship File (PORF)	Establishes relationships between processes and objects, with one record for each relationship between a process and an object.
Process Type File (PTF)	Contains one record for each process type used by ITS processes.
Server Class Type/Process Relationship File (SPRF)	Establishes relationships between server classes and processes, with one record for each relationship between a server class type and a process.
Server Class Type File (SCT)	Contains one record for each server class type used by an ITS process.
Source Class Map File (SCM)	Contains one record for each source class used by an ITS process.

Procedural Processes

The following kernel files contain the kernel configuration for procedural processes (for example, the Enscribe Refresh process) that use the process control mechanism of the ITS environment. Records in these files allow the system to deliver process control commands to a procedural process using an Integrated Server Control File (ISCF).

Process File (PRO)	Contains one record for each procedural process that can receive process control commands.
Process Type File (PTF)	Contains one record for each process type used by a procedural process that can receive process control commands.
Server Class Type/ Process Relationship File (SPRF)	Establishes relationships between server classes and procedural processes that can receive process control commands, with one record for each relationship between a server class type defined in the SCT and a procedural process.
Server Class Type File (SCT)	Contains one record for each server class type used by a procedural process that can receive process control commands. The server class type is used to name the Integrated Server Control File (ISCF) to which process control commands are written.

Logical Network Configuration File

The assigns and params used by kernel modules and objects must be configured in the Logical Network Configuration File (LCONF). The assigns and params used by the Extended Memory Table Build utility are listed in section 5. The assigns and params used by kernel objects are listed by object in the ***BASE24 ITS Kernel Objects Reference Manual***. The assigns and params used by integrated endpoint objects are listed by object in the ***BASE24 IEP Objects Reference Manual***. The assigns and params used by core objects are listed in the application-specific transaction processing manual.

For detailed descriptions of all kernel, integrated endpoint, and core assigns and params, refer to the ***BASE24 Logical Network Configuration File Manual***.

Pathway Configuration File

The Integrated Transaction Services Control Server process provides kernel file maintenance and process control access for the ITS environment and must be configured to the Pathway Configuration File (PATHCONF). Additional procedural servers may also need to be configured to the PATHCONF, depending on the applications used in your ITS environment. The configuration of all procedural servers running under control of the Pathway process for ACI products is provided during installation.

Currently, no ITS processes run under control of the Pathway process.

Network Environment File

All ITS processes and stations that run under control of the XPNET process must be configured in the Network Environment File (NEF) using process or station definitions.

Source Class

The NEF configures the source class for all process and station endpoints in the XPNET node. The SOURCECLASS attribute is used to associate a symbolic destination with a source class. Every process and station definition in your network must have a defined source class. NEFs for all XPNET nodes in an XPNET system are used by the Extended Memory Table Build utility to create the source symbolic map extended memory table. The source symbolic map extended memory table maps process and station symbolic names with the source class configured for the process or station.

Source classes are mapped to message handler objects in the Source Class Map File (SCM). The source classes defined in this file must match the source classes used from the NEF. The Source Class Map File (SCM) extended memory table is created from the Source Class Map File (SCM).

Process Names

The same PPD name and symbolic process name assigned to a process running under control of the XPNET process in the XPNET system, must also be used when configuring an ITS process. Symbolic process names are configured by the ADD PROCESS command. Process PPD names are configured by the PPD attribute in the NEF.

Queue Management

The QMT and QMI attributes in process or station definitions control the network overload management actions for each ITS process and station. Network overload management is a feature of the XPNET process. The QMT attribute specifies the number of messages that can be queued to a process, link, or station before the XPNET process begins generating threshold event messages and evaluating network overload management actions for the queue. The QMI attribute specifies a percentage of XPNET process total memory. If the percentage of total memory

used exceeds the value of this attribute, the XPNET process begins generating threshold event messages and evaluating network overload management actions for the queue.

Normally, the QMT and QMI attributes are used in conjunction with each other, although each can be used independently. If both attributes are used, both thresholds must be exceeded before a threshold message is generated and network overload management actions are evaluated.

The QAT and QED attributes can also be used in ITS process and station definitions to monitor network overload, but no network overload actions in ITS software is based on these attributes. These attributes allow the XPNET process to generate event messages when a user-established threshold is reached.

Additional information on the QMT, QMI, QAT, and QED attributes can be found in the *NET24-XPNET Network Control Operations Guide* and the *NET24-XPNET Network Control Command Reference Manual*.

Network Overload Management Percentage

The XPNET process updates the Network Overload Management Percent field in the message header with overload management information before the message is sent to the ITS process. Based on the settings of the QMT and QMI attributes, the XPNET process calculates two percentages: the percentage for messages queued in relation to the queue management threshold for the queue and the percentage of memory used in relation to the queue management index for the queue. The lower of these two percentages is placed in the Network Overload Management Percent field. For more information on how the XPNET process uses the Network Overload Management Percent field, refer to the *NET24-XPNET Application Programming Guide*.

A value of 0 in the QMT and QMI attributes effectively disables network overload management for a process or station.

Network Overload Percentage Threshold

When an ITS process receives an incoming network data message from a link, process or station, the XPNET Interface object checks the value of the NOM-PERCENT-THRESHOLD param from the LCONF. Valid values for this param are 0 through 100. If this param is set to a value of 0, network overload management processing is not performed and the XPNET Interface object sets the Network Overload Management Percent field in the header to a value of 0. If this

param is set to a nonzero value and the current network overload management percentage in the message header is equal to or exceeds the value of this param, the XPNET Interface object sets this field to a value greater than 100 to indicate that an overload has been detected. If this param is set to a nonzero value and the current network overload management percentage in the message header is less than the value of this param, the object sets this field to a value of 0. Based on the value of this field, the appropriate message handler object (that is, an Endpoint Class Handler object, terminal object, or interface object) can perform the appropriate network overload management actions. A value greater than 100 triggers network overload management actions in the message handler object.

Once a network overload has been detected by the XPNET Interface object, it waits the length of time, in hundredths of seconds, specified by the QED param in the LCONF before reevaluating its queue.

For a detailed description of the NOM-PERCENT-THRESHOLD param, refer to the *BASE24 Logical Network Configuration File Manual*.

Network Overload Management Actions

When network overload management processing is indicated in the network data message header as described above, the message handler object checks the message content to determine whether the message is discarded. Some messages, such as reversals, are never discarded. Messages that are discarded cannot be recovered.

Library Usage

The LIBRARY attribute for all ITS processes must point to the appropriate XPNET library routines. These routines typically reside on your system volume.

The XNLIB.XNLIB file should be used for systems running under XPNET 3.0.

For detailed information on the XPNET library routines, refer to the appropriate informal ACI documentation.

Example Definitions

The following are example definitions for an ITS process and a station.

ITS Process Example

The following is an example of a process definition for an ITS process. The attributes and commands used by the ITS software include the ADD PROCESS command, and the following attributes: LIBRARY, PPD, QMT, QMI, and SOURCECLASS. The process symbolic name and PROGRAM attribute are required for each process definition.

```
SET PROCESS PROGRAM $DATA.OE13OBJ.IEPLO
SET PROCESS LIBRARY $SYSTEM.XNLIB.XNLIB
SET PROCESS CPU 1
SET PROCESS BCPU 2
SET PROCESS STARTUP COMMAND
SET PROCESS PPD $C1E9
SET PROCESS QMT 0
SET PROCESS QMI 0
SET PROCESS SOURCECLASS IEP
SET PROCESS SAVEABEND ON
SET PROCESS PRIORITY 155
ADD PROCESS \SYS1.P1A^NODE.P1A^IEP01
```

ITS Station Example

The following is an example of a station definition for an ITS station. Note that the device and line referenced in the station definition must be added to the node before the station can be added. The ITS software uses the QMI, QMT, and SOURCECLASS attributes. The DEVICENAME, LINENAME, and DESTINATION attributes are required for a station definition.

```
SET STATION DEVICENAME \SYS1.P1A^NODE.IVR^DEVICE
SET STATION LINENAME \SYS1.P1A^NODE.L1A^IVR
SET STATION DESTINATION \SYS1.P1A^NODE.P1A^IEP
SET STATION QMT 0
SET STATION QMI 0
SET STATION SOURCECLASS IVR
SET STATION RETRY 3
SET STATION REINSTATE 10
SET STATION ERRORACT REINEVERY
SET STATION LOGICALACK OFF
ADD STATION \SYS1.P1A^NODE.S1A^VRU
```

Measure Configuration

If you intend to use the Performance Monitoring Subsystem to monitor ITS processing rates, Measure user counters must be established for each ITS process and station to be measured with a counter type of transaction rate. This type of counter measures the number of XPNET process transactions input to and output from a process or station running under control of the XPNET process. In addition, you can also configure queue and state user counters for each Integrated Transaction Server process or station running under control of the XPNET process. All of these counters are configured to a process or station using counter attributes in the process definition.

If rate, queue, or state counters are established for ITS processes and are added to the Performance Monitoring Subsystem configuration files, you can use Performance Monitoring Subsystem facilities to monitor integrated transaction services process and station performance. For detailed information on performance monitoring facilities and procedures, refer to the *NET24 Performance Monitoring Operators Manual*.

Section 3

ITS Configuration Procedures

This section provides step-by-step procedures for configuring the integrated transaction services (ITS) environment. It includes a discussion of process binding, which must be done prior to configuration when the software is installed, and steps for configuring the ITS environment.

Note: Due to the intricate nature of some of these procedures, ACI recommends that ITS environment configuration be performed only by the system manager.

Process Binding Requirements

Some of the kernel objects are required for every object-oriented process running in the ITS environment. These objects must be bound into each ITS process before or during installation and prior to configuration of the ITS environment.

The following kernel objects must be bound into every ITS process:

- Application Network Services object*
- Class object
- Event object
- Process Control object
- Trace object
- XPNET Interface object

* The Space object is currently part of the Application Network Services object.

The Log Permission object must be bound into Log Permission processes only.

Integrated endpoint objects must be bound in to Integrated Endpoint processes only. These objects are described in the ***BASE24 IEP Objects Reference Manual***.

For object-oriented application processes, core objects are bound in to perform specific business functions (for example, electronic funds transfer business functions). These objects are described in the application-specific transaction processing manual.

Process Control Configuration Procedures

Formatted entry screens are provided for easy entry of data into ITS kernel configuration files. To provide referential integrity of file data, some files must be configured before other files. These guidelines show you the correct order in which files should be configured. Some, if not all, of these procedures will be performed when your system is installed.

Prerequisites

Before configuring the ITS environment, your Pathway and XPNET environments should already be configured. The Pathway environment must be configured before you can access ITS kernel file maintenance screens. Some of the data (for example, process symbolic names and source classes) configured in the Process File (PRO) and Source Class Map File (SCM), respectively, of the ITS kernel environment is derived directly from the Network Environment File (NEF) entries for your ITS processes. Although this data can be physically entered in the PRO and SCM before it is configured in the NEF, your ITS processes will not function properly until it is defined in both places.

Assigns can be physically entered in the ITS files before they are entered in the Logical Network Configuration File (LCONF); however, the assigns will not be used until they are configured in both places.

File Relationships and Dependencies

File relationships and dependencies exist among the ITS kernel basic and relationship files. Data must be entered according to these relationships and dependencies. See the ***BASE24 ITS Kernel Files Maintenance Manual*** for a discussion of the file relationships and dependencies that exist for ITS files.

Procedures

To configure the ITS kernel environment, perform the following steps:

1. Enter a record in the Process Type File (PTF) for each object-oriented process type to be used in the ITS system, as well as any process types for procedural processes that use the process control mechanism of the ITS environment (that is, the Integrated Server Control File) for the delivery of commands. A process type must exist in this file before it can be entered in the Process File

(PRO), Assign/Process Type Relationship File (APRF), or the Extended Memory Table/Process Type Relationship File (EPRF). Note that some process types are preconfigured by ACI and do not require data entry. The file maintenance screen for the Process Type File (PTF) is described in detail in the ***BASE24 ITS Kernel Files Maintenance Manual***.

Under certain circumstances, you may be required to modify the contents of the PTF. Generally, the changes required will be adding information to the file rather than deleting data from the file. See the “Modifying Default Data” discussion provided later in this section.

2. Enter records in each of the following ITS basic files:

- Process File (PRO)
- Assign File (ASGN)
- Object File (OBJ)
- Extended Memory Table File (EMT)
- Server Class Type File (SCT)

Each of the file maintenance screens for these files is described in detail in the ***BASE24 ITS Kernel Files Maintenance Manual***.

Note that some of these files will contain default data that has been preconfigured by ACI and will not require data entry. In all likelihood, the following files will be completely preconfigured and will require no configuration whatsoever on your part.

- Assign File (ASGN)
- Object File (OBJ)
- Extended Memory Table File (EMT)

Under certain circumstances, you may be required to modify the contents of some of these files. Generally, the changes required will be adding information to the files rather than deleting from them. See the “Modifying Default Data” discussion provided later in this section.

In most cases, the only data you will have to enter for this step will be in the PRO and SCT. These files cannot be defined in advance because they contain references to process symbolic names, which are always site-specific.

The following information should be available prior to configuring the PRO and SCT:

- A listing of the current XPNET system configuration. You can use the INFO command with the OBEYFORM modifier to generate a listing of the current configuration. Note that this command does not list

attributes that currently have default values. For more information about the OBEYFORM modifier, see the topic “Generating a List of the Current Configuration” in the *NET24-XPNET Network Control Operations Guide* or the descriptions of the INFO commands in the *NET24-XPNET Network Control Command Reference Manual*.

- MAKE files for each process type. The following table shows you the location and file name of the MAKE file for each type of object-oriented ITS process. These MAKE files identify all objects used to make up the process executable. Some objects are used conditionally, depending on the ITS environment settings in the *\$vol.BA53MAKE.CUSTMACS* file.

Make File	Process Type
<i>\$vol.OC13LINK.EOPPLM</i>	End of Period process
<i>\$vol.OC13LINK.FCLM</i>	Fraud Control process
<i>\$vol.OC13LINK.IASLM</i>	BASE24-telebanking Integrated Authorization Server process
<i>\$vol.OC13LINK.IMPLM</i>	Interface Master process
<i>\$vol.OE13LINK.IEPLM</i>	Integrated Endpoint process
<i>\$vol.OK13LINK.LPPLM</i>	Log Permission process

If MAKE files are not available, you can use Tandem’s Binder product to determine the objects that make up an executable process (known as an object). All executable object files have a file name that ends in LO and can be found in the object subvolume for your system (for example, *\$vol.PRO1OBJ*). For detailed information on Binder, refer to the Tandem *Binder Manual*.

Objects must be related to each process in which they are used in the Process/Object Relationship File (PORF). See step 3b.

- Configure the Process File (PRO). A record for each ITS process in the XPNET system must be defined in the PRO. The entries in this file should include all ITS processes that can receive TRACE, DEBUG, and STATUS commands from the Process Control subsystem. A record must also be defined for each procedural process that uses the process control mechanism of the ITS environment.

For each ITS process in the XPNET system, perform the following steps:

- 1) Enter the process symbolic name in the PROCESS NAME field on PRO screen 1.

- 2) Enter the process PPD name in the PPD NAME field on PRO screen 1.

All ITS processes must be named processes. A named process has a PPD name assigned to it in the XPNET system. Processes must be named so that a process control command can be targeted to a specific process within a server class.

- 3) Set the QUEUE FLAG field on PRO screen 1 to the value N. This field is not currently used.
- 4) Set the process type in the PROCESS TYPE field on PRO screen 1, accordingly. The following process types have been defined.

Process Type	Description
EOPP	End-of-period process. A process that performs settlement processing at the end of a reporting period.
IAS	Integrated Authorization Server process. Any ITS process that performs authorization logic. Note that the Fraud Control process is an Integrated Authorization Server process.
IEP	Integrated Endpoint process. Any ITS process that contains an Endpoint Class Handler object.
IMP	Interface Master process. Any ITS process that performs interface station management.
LPP	Log Permission process. Any ITS process that controls event suppression at the network level.
REFRESH	Enscribe Refresh process. A procedural process that performs refresh processing of Enscribe files.

The process type is typically used as part of the executable object file names that end in LO (for example, IASLO, IEPLLO). The only exceptions are the Fraud Control process, which has an executable object file name of FCLO, and the Enscribe Refresh process, which has an executable object file name of REFRLO.

An example XPNET process definition is shown below. In this example, the process has a process type of IEP. The information relevant to PRO configuration is displayed in bold print.

```

SET PROCESS PROGRAM $DATA.OE13OBJ.IEPLO
SET PROCESS LIBRARY $SYSTEM.XNLIB.XNLIB
SET PROCESS CPU 1
SET PROCESS BCPU 2
SET PROCESS STARTUP COMMAND
SET PROCESS PPD $C2E0
SET PROCESS QMT 0
SET PROCESS QMI 0
SET PROCESS SOURCECLASS IEP
SET PROCESS SAVEABEND ON
SET PROCESS PRIORITY 165
ADD PROCESS \SYS1.P1A^NODE.P2A^IEP01

```

For the example above, PRO screen 1 would be completed as shown below.

```

BASE24-KNL    PROCESS FILE                PRO1          YY/MM/DD  HH:MM  01 OF 01

```

```

PROCESS NAME:P2A^IEP01

```

```

PPD NAME:$C2E0

```

```

PROCESS TYPE:IEP      INTEGRATED END POINT

```

```

QUEUE FLAG: N

```

```

***** BASE24 *****

```

```

NEW PAGE:          FILE DESTINATION:          NEW LOGICAL NETWORK ID:
                  F12-HELP

```

b. Configure the Server Class Type File (SCT).

After all processes and queues are configured in the PRO, you must determine which server classes are to be defined. A server class is a means of organizing processes into logical groups for process control. An SCT entry must be defined for each server class created.

Note: An SCT record must also be defined for each server class of procedural processes (for example, the Enscribe Refresh process) that use the process control mechanism of the ITS environment.

Processes in a server class share an Integrated Server Control File (ISCF). The ISCF is the mechanism used to deliver process control commands to processes within a server class. **All processes** look at **every** command posted into their respective ISCF. Therefore, it is a good idea to partition server classes across natural command boundaries. Improper designation of server classes could cause unwanted cross talk between separate applications. For example, placing Integrated Endpoint processes in a server class separate from Integrated Authorization Server processes prevents application changes from disrupting the Integrated Endpoint processes. Placing ITS processes for one application in a separate server class from ITS processes for another server class prevents commands to one application from disrupting service to other applications.

For each server class to be defined, perform the following:

- 1) Enter the server class type in the SERVER CLASS TYPE field. The name entered here is also used in the Server Class Type/ Process Relationship File (SPRF).
- 2) Enter a descriptive name for the server class in the SERVER CLASS TYPE DESCRIPTION field. This field is for informational purposes only and should be descriptive of the functions performed by the server class.
- 3) Enter a control file assign name for the Integrated Server Control File (ISCF). This must be an ISCF assign specified for the server class in the LCONF. All processes within the server class will use this assign to find their ISCF.

The example SCT entry shown below defines a server class called IAS TELEBANKING for the BASE24-telebanking application. The ISCF used by servers in this class is ISCF-IAS. The control number is automatically set to 0 when the SCT entry is added.


```
BASE24-KNL  SERVER CLASS TYPE FILE  PRO1      YY/MM/DD  HH:MM  01 OF 01
```

```
SERVER CLASS TYPE: IAS TELEBANKING
```

```
SERVER CLASS TYPE DESCRIPTION: TELEBANKING SERVERS
```

```
CONTROL FILE ASSIGN NAME: ISCF-IAS
```

```
CONTROL FILE NUMBER: 0
```

```
***** BASE24 *****
NEW PAGE:          FILE DESTINATION:      NEW LOGICAL NETWORK ID:
                  F12-HELP
```

3. Enter records in each of the following ITS relationship files. The order in which these files are configured is unimportant as long as they are configured after the basic files they relate have been configured.
 - Server Class Type/Process Relationship File (SPRF)
 - Process/Object Relationship File (PORF)
 - Log Permission Process Partition File (LPF)
 - Assign/Process Relationship File (APRF)
 - Assign/Object Relationship File (AORF)
 - Source Class Map File (SCM)
 - Extended Memory Table/Process Type Relationship File (EPRF)
 - Extended Memory Table/Object Relationship File (EORF)

Each of the file maintenance screens for these files is described in detail in the ***BASE24 ITS Kernel Files Maintenance Manual***.

Note that some of these files will contain default data that has been preconfigured by ACI and will not require data entry. In all likelihood, the following files will be completely preconfigured and will require no configuration whatsoever on your part.

- Assign/Object Relationship File (AORF)
- Assign/Process Relationship File (APRF)
- Extended Memory Table/Object Relationship File (EORF)
- Extended Memory Table/Process Type Relationship File (EPRF)

Under certain circumstances, you may be required to modify the contents of some of these files. Generally, the changes required will be adding information to the file rather than deleting from the file. See the “Modifying Default Data” discussion provided later in this section.

In most cases, the only data you will have to enter will be in the LPF, SCM, SPRF and PORF. For the most part, these files cannot be defined in advance because they contain references to site-specific information such as process symbolic names.

- a. Configure the Server Class Type/Process Relationship File (SPRF). This file relates server class types configured in the SCT with processes configured in the PRO. The processes belonging to each server class are defined in this step.

Note: Any server classes of procedural processes (for example, the Enscribe Refresh process) that use the process control mechanism of the ITS environment must also be configured in the SPRF.

The example below depicts process P7A^IAS01 belonging to the TELEBANKING server class for the BASE24-telebanking application. The server class type and process name are checked against the Server Class Type File (SCT) and Process File (PRO), respectively. Entries for the server class type and process name must exist in these files prior to being related in the SPRF.

```

BASE24-KNL  SRV CLASS/PRO RELATION  PRO1          YY/MM/DD  HH:MM  01 OF 01

                SERVER CLASS TYPE: IAS TELEBANKING      (TELEBANKING SERVERS)

                PROCESS NAME: P7A^IAS01

```

```

***** BASE24 *****
NEW PAGE:          FILE DESTINATION:          NEW LOGICAL NETWORK ID:
                  F12-HELP

```

- b. Configure the Process/Object Relationship File (PORF). Once processes have been defined using the PRO and objects have been defined using the OBJ, process and object relationships can be defined in the PORF. A PORF entry must be made for every object contained in every ITS process in the network.

For each process and object relationship, perform the following:

- 1) Enter the process symbolic name in the PROCESS NAME field on PORF screen 1. The name entered is verified against the PRO.
- 2) Enter the object name in the OBJECT NAME field on PORF screen 1. The object name is verified against the OBJ.

The example below defines the Event object as a component of the P7A^IAS01 process.

```
BASE24-KNL    PROCESS/OBJECT RELATION  PRO1          YY/MM/DD  HH:MM  01 OF 01
```

```
PROCESS NAME: P7A^IAS01
OBJECT NAME:  EVT          EVENT OBJECT
```

```
***** BASE24 *****
NEW PAGE:          FILE DESTINATION:  NEW LOGICAL NETWORK ID:
                  F12-HELP
```

- c. Configure the Source Class Map File (SCM) for each source class in your system from which messages can be received. See the ***BASE24 ITS Kernel Files Maintenance Manual*** for a detailed description of the SCM file maintenance screen and its fields. The type of process or station associated with a source class dictates the object that handles messages received from that process or station as shown in the following table.

Type of Process or Station	Message Handler Object
Procedural	Internal Message Translate object (IMTT)
Object-oriented	XPNET Interface object (XPN)
Endpoint-specific (for example, a remote banking endpoint device station)	Endpoint-specific Endpoint Class Handler object (for example, VRUE for a remote banking endpoint)

Type of Process or Station	Message Handler Object
Interface-specific (for example, a customer service representative interface)	Interface-specific object (for example, the Customer Service Interface object (CSI) for the customer service representative interface)

An example of the process definition for an object-oriented process is shown below. In this example, the source for this process from which messages are received is IEP.

The information relevant to SCM configuration is displayed in bold print.

```
SET PROCESS PROGRAM $DATA.OE13OBJ.IEPLO
SET PROCESS LIBRARY $SYSTEM.XNLIB.XNLIB
SET PROCESS CPU 1
SET PROCESS BCPU 2
SET PROCESS STARTUP COMMAND
SET PROCESS PPD $C2E0
SET PROCESS QMT 0
SET PROCESS QMI 0
SET PROCESS SOURCECLASS IEP
SET PROCESS SAVEABEND ON
SET PROCESS PRIORITY 165
ADD PROCESS \SYS1.P1A^NODE.P2A^IEP01
```

The example below depicts the SCM record for the source class IEP.

```
BASE24-KNL    SOURCE CLASS MAP FILE    PRO1        YY/MM/DD  HH:MM  01 OF 01
```

```
SOURCE CLASS: IEP
```

```
OBJECT NAME: XPN (<object description>)
```

```
***** BASE24 *****
NEW PAGE:          FILE DESTINATION:    NEW LOGICAL NETWORK ID:
                   F12-HELP
```

4. Configure any files or tables used to build extended memory tables for the ITS environment. Files used to build kernel extended memory tables are described in section 5. Files used to build IEP extended memory tables are described in the ***BASE24 IEP Configuration Manual***. Files and tables used to build core extended memory tables are described in the application-specific transaction processing manual.
5. Build any shared extended memory tables to be used for your ITS environment using the Extended Memory Table Build utility. Extended memory tables that are not shared are built by the object that uses them when the object is initialized. The Extended Memory Table Build utility can be run from the Configuration Control screen (CCS) or directly from a TACL prompt. Detailed instructions for building extended memory tables are provided in section 4 of this manual. All extended memory tables that can be reallocated using the EMT WARMBOOT command must also be configured in the Extended Memory Table/Object Relationship File (EORF) and the Extended Memory Table/Process Type Relationship File (EPRF).
6. Configure the integrated endpoint processing for your ITS environment. For detailed Integrated Endpoint process configuration considerations and instructions, refer to the ***BASE24 IEP Configuration Manual***.

Modifying Default Data

Under certain circumstances you may be required to modify the contents of default data provided by ACI. Generally speaking, the changes required involve adding information to the files rather than deleting information from them. Two of the most common conditions that require customer modification of default data are discussed below.

- You want to change the text descriptions for certain entities. You may do this because you prefer other descriptions or because you want to convert the descriptions to a language other than U.S. English.
- You add additional assigns, extended memory tables, objects, or processes to your system as a result of your own development or as a result of CSMs developed by ACI.

The remainder of this subsection discusses various procedures for adding information beyond the preconfigured default data provided by ACI.

Adding New Assigns

To add a new assign to an ITS system, perform the following steps. Refer to the ***BASE24 ITS Kernel Files Maintenance Manual*** for detailed descriptions of the kernel file maintenance screens used in these steps.

1. Add the assign to the Logical Network Configuration File (LCONF). For detailed instructions on defining LCONF assigns, refer to the ***BASE24 Logical Network Configuration Manual***.
2. If the assign can be reread when an object receives the ASSIGN WARMBOOT command, perform the following:
 - a. Add a record for the assign to the Assign File (ASGN).
 - b. Add a record to the Assign/Object Relationship File (AORF) for each object that can reread the assign.
 - c. Add a record to the Assign/Process Type Relationship File (APRF) for each process type containing the object that can reread the assign.

Adding New Extended Memory Tables

To add a new extended memory table to an ITS system, perform the following steps.

Refer to the *BASE24 ITS Kernel Files Maintenance Manual* for detailed descriptions of the kernel file maintenance screens used in these steps.

1. Add a record for the extended memory table to the Extended Memory Table File (EMT).

If an extended memory table cannot be built by the Extended Memory Table Build utility or be reallocated using the EMT WARMBOOT command, it should not be configured in the EMT.
2. If the extended memory table can be reallocated when an object receives the EMT WARMBOOT command, perform the following:
 - a. Add a record to the Extended Memory Table/Object Relationship File (EORF) for all objects that can reallocate the extended memory table.
 - b. Add a record to the Extended Memory Table/Process Type Relationship File (EPRF) for any process types containing objects that can reallocate the extended memory table.
3. If extended memory table is shared, build the table using the Extended Memory Table Build utility. Refer to section 4 of this manual for instructions on using the Extended Memory Table Build utility.
4. Execute the EMT WARMBOOT command or LCONF WARMBOOT command on any objects that use the extended memory table to allocate and read the table. The LCONF WARMBOOT command must be used for extended memory tables that do not allow the EMT WARMBOOT command.

Adding New Objects

To add a new object to an ITS system, perform the following steps.

1. Add a record for the object to the Object File (OBJ). See the ***BASE24 ITS Kernel Files Maintenance Manual*** for details on the OBJ file maintenance screen and its fields. When adding an object, you must select an object name and object ID. These values are generally selected prior to development of the object. See the field descriptions for the OBJ file maintenance screen for detailed information on object names and object IDs.
 - a. Add an object ID for each object created to the appropriate OBJIDDS file in your ITS system to properly register the object to the ITS system.

Kernel object IDs are added to the *\$vol.OK13DDL.KOBJIDDS* file. IEP object IDs are added to the *\$vol.OE13DDL.EOBJIDDS* file. Core object IDs are added to the *\$vol.OC13DDL.COBJIDDS* file.
 - b. Define the object name in the OBJECT NAME field on OBJ screen 1. Object names can be up to four characters in length. A descriptive acronym is generally used and the name is typically used as part of the object's source code subvolume naming conventions.
 - c. Add a descriptive name for the object in the OBJECT DESCRIPTION field on OBJ screen 1.
 - d. Next, select the appropriate object types from those defined for the object in the related TAL Globals (xxTG) file in the appropriate object source subvolume, where xx is set to either OK for kernel objects, OE for integrated endpoint objects, or OC for core objects.

The kernel TAL Globals file for kernel objects resides in the OK13SRC subvolume, in the OE13SRC subvolume for integrated endpoint objects, and in the OC13SRC subvolume for core objects.

Object type values are single-character values ranging from A–Z and 0–9 identifying the traits of an object. The following table shows you the object type ranges currently defined by ACI. No object type values are currently defined for integrated endpoint objects. The values A and 0 through 8 have already been defined by ACI and are reserved. If you need to define additional traits, you must use a value that has not already been reserved. Each of the values defined by ACI are described with the OBJECT TYPE field description for the OBJ in the ***BASE24 ITS Kernel Files Maintenance Manual***.

Object Classification	Start of Range	End of Range
Kernel Objects	A	F
Core Objects	G	9

Enter the selected object types in the OBJECT TYPE fields on OBJ screen 1.

- e. If the object supports the LCONF WARMBOOT command, set the WARMBOOT LCONF FLAG field on OBJ screen 1 to a value of Y. Otherwise, set this field to a value of N.
 - f. If the object supports the PARAM WARMBOOT command, set the WARMBOOT PARAM FLAG field on OBJ screen 1 to a value of Y. Otherwise, set this field to a value of N.
 - g. If the object supports any general commands (that is, Server Control commands) entered from the Server Control screen (SCS), set the GENERAL COMMAND FLAG field on OBJ screen 1 to a value of Y. Otherwise, set this field to a value of N.
 - h. If the object supports the STATUS command entered from the Status screens (STAT), set the STATUS COMMAND FLAG field on OBJ screen 1 to a value of Y. Otherwise, set this field to a value of N.
 - i. Press the **F9** key to view OBJ screen 2, set the trace levels as needed for the object on OBJ screen 2, and press the **F3** key to add the OBJ record. Note that trace levels 33–64 are reserved for future use and should not be used.
2. Add any new assigns or params for the object to the LCONF. See the ***BASE24 Logical Network Configuration File Manual*** for directions on entering LCONF assigns and params.
 3. Add records for any assigns that can be reread or extended memory tables that can be reallocated to the Assign File (ASGN) and Extended Memory Table File (EMT), respectively. See the ***BASE24 ITS Kernel Files Maintenance Manual*** for details on both the ASGN and EMT file maintenance screens.
 4. Add all required relationship records for the object in the Assign/Object Relationship File (AORF), the Extended Memory Table/Object Relationship File (EORF), and the Process/Object Relationship File (PORF). See the ***BASE24 ITS Kernel Files Maintenance Manual*** for details on the AORF, EORF, and PORF file maintenance screens and their fields.
 5. Add a subsystem identifier (SSID) for the object to the \$volume.SCRIBE. EMCLDDLS file. Subsystem identifiers are used by the Event Management Service (EMS) to identify events generated by an object and are also used

when configuring event suppression for an object in the ESF and LPF. To reduce the possibility of duplicated names, the SSID literal names used throughout ACI adhere to the following convention:

ACI-SSN-productsubsystem-L

Where:

The *product* is four characters that identify the product to which the SSID belongs. Kernel SSIDs use a product identifier of ISKN, integrated endpoint SSIDs use a product identifier of ISIE, and core SSIDs use a product identifier of ISCO.

The *subsystem* is up to four characters identifying an object within the product. For example, the ACI-SSN-ISKNANS-L literal identifies events generated by the Application Network Services (ANS) object, which is a kernel object. Note that there is no space between the product and subsystem identifiers.

A numeric value is associated with the literal in the EMCLDDLS file.

When adding a customer-specific SSID, perform the following steps:

- a. Select an eight-character SSID literal name as described above.
- b. Volume to the *\$volume*.SCRIBE subvolume, where *\$volume* is the name of the disk volume where the SCRIBE.EMCLDDLS file resides.
- c. Obey the GOADD file. This file will prompt you for the eight-character SSID literal name selected in step 5a. This file will also prompt you for a three-character abbreviation for the SSID being entered. You can use any three characters desired.

```
TACL> O[BEY] GOADD
```

- d. Repeat step 5c for every SSID to be added.
- e. Once all the SSIDS have been entered, obey the GOINST file to update the files located in the SCRIBE subvolume pertaining to the EMS subsystem. Check the output report on the spooler for any errors that may have occurred.

```
TACL> O[BEY] GOINST
```

- f. Copy the EMDDLTLAL file to the EMCLTLAL file using the Tandem File Utility Program (FUP). Both files reside in the *\$volume*.SCRIBE subvolume.

```
TACL> FUP DUP EMDDLTLAL, EMCLTLAL
```

- g. Once the EMCLTAL file has been replaced with the EMDDLTLAL file, edit the EMCLTAL file and replace all circumflex characters (^) with the underbar character (_).

```
TACL> EDIT EMCLTAL
*CAB/^/_/a
```

Section 4

Extended Memory Table Build Utility

This section provides detailed instructions for using the Extended Memory Table Build utility. This utility is a stand-alone program used to build or rebuild shared extended memory tables from disk files or tables. The Extended Memory Table Build utility can be executed from the Configuration Control process control screen (CCS) or from a TACL prompt. This section discusses the following topics:

- An overview of Extended Memory Table Build utility processing, including procedures to make processes or objects to reallocate a newly built or rebuilt extended memory table
- A description of the source files from which extended memory tables are created for the integrated transaction services (ITS) kernel
- The configuration requirements for the Extended Memory Table Build utility, including a list of all Logical Network Configuration File (LCONF) assigns and params
- Procedures for starting the Extended Memory Table Build utility from the Configuration Control screen (CCS) or from a TACL prompt
- A description of the Extended Memory Table Build Report produced when the Extended Memory Table Build utility is run

Overview

The Extended Memory Table Build utility builds shared extended memory tables for use by ITS application objects and processes. The program can build hash tables, radix tables, or custom-designed tables.

Shared extended memory tables are built in the background while an ITS application is processing.

The Extended Memory Table Build utility currently supports the following shared extended memory tables for the kernel environment.

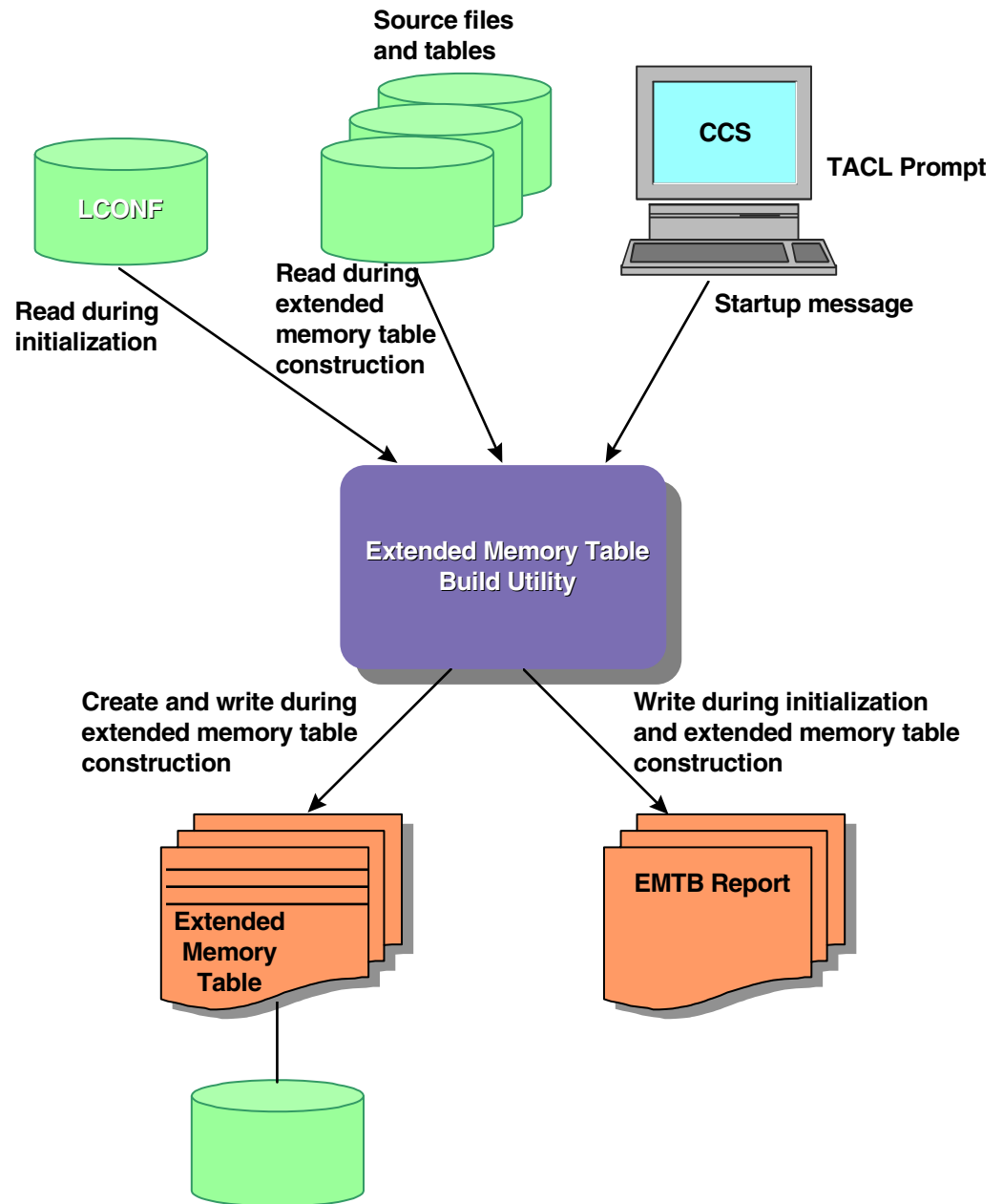
- Event Suppression File (ESF) extended memory table
- Source Class Map File (SCM) extended memory table
- Source symbolic map extended memory table

For information on extended memory tables supported by the Extended Memory Table Build utility for integrated endpoint objects, refer to the ***BASE24 IEP Configuration Manual***. For information on extended memory tables supported by the Extended Memory Table Build utility for core objects, refer to the product-specific transaction processing manual.

The Extended Memory Table Build utility creates an Extended Memory Table Build report summarizing the processing performed each time it is executed. The Extended Memory Table Build report is described later in this section.

Extended memory tables are configured to the ITS environment using the Extended Memory Table File (EMT), the Extended Memory Table/Object Relationship File (EORF), and the Extended Memory Table/Process Type Relationship File (EPRF).

The processing performed by the Extended Memory Table Build utility is depicted in the following illustration. A key to the abbreviations follows the illustration.



Key

CCS = Configuration Control screen
EMTB = Extended Memory Table Build utility
LCONF = Logical Network Configuration File

Allocating a Newly Built or Rebuilt Extended Memory Table

Once an extended memory table is built or rebuilt, it will not be used until the application that uses the extended memory table has been informed of the change and reallocates the table. The procedures for informing processes vary according to the type of application process (that is, object-oriented or procedural) that uses the table and the environment (for example, XPNET or Pathway) in which the process runs.

Object-Oriented Processes

Extended memory tables are reread by object-oriented processes using the EMT WARMBOOT command and the LCONF WARMBOOT command. The EMT WARMBOOT command is entered from the EMT Warmboot (EWB) screen and allows individual objects to reread an individual extended memory table. When an object receives the EMT WARMBOOT command, it reallocates the table if it is shared and rebuilds the table if it is not shared. This command is only allowed if the EMT WARMBOOT ALLOWED field on EMT screen 1 is set to a value of Y for the extended memory table. If not, you must use the LCONF WARMBOOT command. The LCONF WARMBOOT command is entered from the LCONF Warmboot screen (LWB) and causes the entire process to reinitialize, including the allocation of extended memory tables. For detailed information on the EWB and LWB screens, refer to the *BASE24 ITS Kernel Files Maintenance Manual*.

The EMT WARMBOOT command causes an object to validate the name of the extended memory table, allocate the new extended memory table, deallocate the old extended memory table, and replace the extended memory table segment ID in the frame. If more complicated processing is required for an extended memory table (for example, building a secondary frame), then you must use the LCONF WARMBOOT command to reallocate the table.

Procedural Processes

Procedural processes that use extended memory tables reread extended memory tables using the warmboot facilities provided for the environment in which they run—Pathway or XPNET. Use of these facilities is described in the following paragraphs.

Pathway Environment. To cause a procedural server process running under control of the Pathway process to reread an extended memory table, perform the following:

1. Start a Pathcom session from a TACL prompt.
`TACL> PATHCOM pathmon-process-name`
2. From the Pathcom prompt, freeze the server process.
`= FREEZE SERVER server-name`
3. Stop the server process.
`= STOP SERVER server-name`
4. Thaw the server process.
`= THAW SERVER server-name`
5. Start the server process.
`= START SERVER server-name`

When the server is started it will reinitialize, causing the server to reallocate the extended memory table. If the server was configured as a dynamic server, you will receive an error message which can be ignored.

For more information on PATHCOM commands, refer to the *NonStop TS/MP System Management Manual*.

XPNET Environment. To cause a procedural process running under control of the XPNET process to reallocate an extended memory table, perform the following:

1. Access a network control facility (for example, the NCPcom utility or the Network Control Supervisor screen).
2. Stop and start the process or warmboot the process.
 - a. To stop and start a process, enter the following commands from a network control facility.

```
STOP PROCESS sym-proc-name
START PROCESS sym-proc-name
```

For detailed syntax information for these commands, refer to the *NET24-XPNET Network Control Command Reference Manual*.

- b. To warmboot a process, enter the appropriate DELIVER PROCESS WARMBOOT (9501) command for the target process. The exact syntax of the command may vary according to the type of process for which the command is intended. For detailed information on this command, refer to the *BASE24 Text Command Reference Manual*.

Source Files and Tables

Extended memory tables must be rebuilt anytime a change is made to the source files and tables for the extended memory table. The source files and tables for kernel extended memory tables are identified below. Source files and tables for integrated endpoint extended memory tables are identified in the *BASE24 IEP Configuration Manual*. Source files and tables for core extended memory tables are identified in the product-specific transaction processing manual. Source files and tables for extended memory tables used by procedural processes are identified in the appropriate product-specific manual or in the *BASE24 Logical Network Configuration File Manual*.

Event Suppression File Extended Memory Table

The Event Suppression File (ESF) extended memory table must be rebuilt anytime a change is made to the Event Suppression File (ESF). The ESF contains one record for each event that is to be suppressed in some way.

Source Class Map File Extended Memory Table

The Source Class Map File (SCM) extended memory table must be rebuilt anytime a change is made to the Source Class Map File (SCM). The Source Class Map File is used to relate source class (SOURCECLASS attribute) values defined in the XPNET system with message handler objects in the ITS environment.

Source Symbolic Map Extended Memory Table

The source symbolic map extended memory table must be rebuilt anytime a change is made to a station or process in the XPNET system from which ITS messages can originate. The source symbolic map extended memory table is used to associate source symbolic names with a source class for processes or stations from which messages can originate.

LCONF Assigns and Params

The following LCONF assigns and params are used by the Extended Memory Table Build utility on behalf of kernel objects or modules. Other assigns and params used by the utility on behalf of the Integrated Endpoint process and core products are listed in the *BASE24 IEP Configuration Manual* and the product-specific transaction processing manual, respectively.

Each of these assigns and params is described in detail in the *BASE24 Logical Network Configuration File Manual*.

Assigns	Params
EMT-SWAPVOL	ALT-TIM-LMT
ESEMT	CONSOLE-PRT
ESF	DATE-DISPLAY-FORMAT
NETLIST-PROG	NETLIST-CPU
OBJ	NETLIST-PRI
SCM	PRI-TIM-LMT
SCMEMT	CPFEMT-MAX-TBL-SIZE
SPANCTL	CUSTEMT-MAX-TBL-SIZE
SSMEMT	DESTEMT-MAX-TBL-SIZE
	ESEMT-MAX-TBL-SIZE
	FPFEMT-MAX-TBL-SIZE
	IDFEMT-MAX-TBL-SIZE
	IMTEMT-MAX-TBL-SIZE
	INTFEMT-MAX-TBL-SIZE
	IRTEMT-MAX-TBL-SIZE
	KEYAEMT-MAX-TBL-SIZE
	SCMEMT-MAX-TBL-SIZE
	SSMEMT-MAX-TBL-SIZE
	THFEMT-MAX-TBL-SIZE
	TLFEMT-MAX-TBL-SIZE

Startup

This subsection describes the procedures for starting the Extended Memory Table Build utility from a TACL prompt. You can run the utility at a high or low process identification number (PIN) when you start it from a TACL prompt. Instructions for starting the Extended Memory Table Build utility using the Configuration Control (CCS) screens are provided in the *BASE24 ITS Kernel Files Maintenance Manual*. The utility always runs at a low PIN when started from the CCS screens. For more information on high and low PINs, refer to section 1.

The startup message used to start the Extended Memory Table Build utility from a TACL prompt defines the extended memory tables which are to be built. The format of the Extended Memory Table Build startup message sent to the Extended Memory Table Build program is derived from the RUN command shown below.

Syntax

```
[ RUN ] [ $location ] EMTBLO/ [ HIGHPIN { ON | OFF } ] [ OUT $printer/ ]  
LCONF file-name, emt-name [ ,emt-name ]
```

RUN — An optional keyword for the RUN command. If this keyword is included in the command, either the Extended Memory Table Build utility must be located on the current volume and subvolume or the location variable must be included in the command. If this keyword is not included in the command, the Extended Memory Table Build utility must be located on a volume and subvolume included in your PMSEARCHLIST.

\$location — Specifies the location of the Extended Memory Table Build utility. This variable, which can include the volume, subvolume, or both, is required if the Extended Memory Table Build utility is not on the current volume or subvolume and the location of the Extended Memory Table Build utility is not included in your PMSEARCHLIST. If this variable is included in the command, the keyword RUN must also be included in the command. The Extended Memory Table Build utility is typically located on the OC13OBJ subvolume.

HIGHPIN — An optional variable for the RUN command. If this variable is set to the value ON, the parent TACL process runs at a high PIN, and other HIGHPIN options are set appropriately, the utility will run at a high PIN. If this variable is not included or is set to the value OFF, and other HIGHPIN options are set appropriately, the utility will run at a low PIN. For more information on how different HIGHPIN options affect whether a new process runs at a high or low PIN, refer to the *TACL Reference Manual*.

printer — Specifies the spooler location to which reports are sent. If you do not specify a spooler location, it defaults to \$S.#HOLD.

file-name — Specifies the fully qualified name of the Logical Network Configuration File (LCONF) containing the Extended Memory Table Build utility assigns and params.

emt-name — Specifies the name of the extended memory table to be built. The RUN statement can contain any number of extended memory table names, from a single extended memory table name to all of the valid extended memory table names. Each name must be separated by a comma.

The valid names for extended memory tables currently used by ACI products are as follows:

CPFEMT	= Card Prefix File extended memory table
CUSTEMT	= Customer extended memory table
DESTEMT	= Destination File extended memory table
ESEMT	= Event suppression extended memory table
FPFEMT	= Fraud Parameters File extended memory table
IDFEMT	= Institution Definition File extended memory table
IMTEMT	= Internal Message Translation File extended memory table
INTFEMT	= Interface extended memory table
IRTEEMT	= Institution routing extended memory table
ISECEMT	= Interface Security File extended memory table
KEYAEMT	= Key Authorization File extended memory table
SCMEMT	= Source Class Map File extended memory table
SSMEMT	= Source symbolic map extended memory table
THFEMT	= Transaction history file extended memory table
TLFEMT	= Transaction log file extended memory table

Example

```
RUN \b24.$data.oc13obj.EMTBLO/HIGHPIN ON, OUT $s.#emt/LCONF
\b24.$data.pro1exec.l1conf, CUSTEMT, IDFEMT, IRTEEMT
```

This example command builds the customer extended memory table, the Institution Definition File extended memory table, and the institution routing extended memory table. The Extended Memory Table Build utility runs at a high PIN and reports are sent to the \$S. #EMTB spooler location.

Extended Memory Table Build Report

The Extended Memory Table Build Report provides a summary of all processing performed by the Extended Memory Table Build utility. A sample of the Extended Memory Table Build Report is shown below and on the following pages. Each sample page is followed by an explanation of each field appearing on the report page.

The title of the report and the current page number are printed at the top of each report page.

In addition to the fields documented here, various self-explanatory exception messages may be written to the report at any time indicating exceptions that occur during processing. If an exception causes the build utility to abend, the phrase “*emt-name* BUILD HALTED” will be written to the report, where *emt-name* is the name of the extended memory table being built when the exception occurs.

Cover Page

The cover page is printed at the beginning of the Extended Memory Table Build report. It summarizes the start date and time and lists the extended memory tables that were requested to be built.

BASE24	EXTENDED MEMORY TABLE BUILD REPORT	PAGE: 1
Start Date:	YY/MM/DD	
Start Time:	HH:MM:SS	
EMT Swap Files Requested:		
	SCMEMT	
	SSMEMT	

Start Date — The date, in the format specified by the DATE-DISPLAY-FORMAT LCONF param, the Extended Memory Table Build utility began processing.

Start Time — The time of day (HH:MM:SS) the Extended Memory Table Build utility began processing.

EMT Swap Files Requested — The names of the extended memory tables requested to be built.

Hash Table Summary Page

A Hash Table Summary report page is printed for each extended memory table built with the exception of some core extended memory tables that have their own report page format. For descriptions of any core-specific report pages, refer to the product-specific transaction processing manual.

The Hash Table Summary page details the hashing algorithms and bucket factors used to build the optimum hash table for a file or table.

BASE24	EXTENDED MEMORY TABLE BUILD REPORT	PAGE: <i>n</i>
Table Type:	SCMEMT	
Source File:	\SYS.\$REL5D.PRO1DATA.SCM	
EMT Swap File:	\SYS.\$REL5D.PRO1DATA.SCMEMT	
Table Size:	135470 bytes	
Total Data Records:	13	
Maximum Record Length:	90 bytes	
Key Type:	ALL STRING	
Key Length:	16 bytes	
Key Algorithm:	DOUBLE BYTE SUMMATION	
Hash Algorithm:	MODULO DIVIDE	
Bucket Factor:	2	
Total Buckets:	29	
Total Buckets Used:	13	
Percent Buckets Used:	44%	
Total Barrels:	1	
Total Collisions:	0	
Percent Collisions:	0%	
Total Collision Chains:	0	
Longest Collision Chain:	0	
Average Collision Chain:	0	
Build Attempts:	24	

Table Type — The acronym for the extended memory table being built.

Source File — The fully qualified location of the source file or table on disk from which the extended memory table is to be built.

EMT Swap File — The fully qualified location of the swap file used to build this extended memory table. This is the name of the actual extended memory table file on disk. This file is opened by the object when it allocates the extended memory segment. The swap file is used to swap out records from disk as they are added to the table in memory.

Table Size — The total size, in bytes, required for the extended memory table.

Total Data Records — The total number of data records or rows included in the extended memory table.

Maximum Record Length — The maximum length, in bytes, of each record included in the extended memory table.

Key Type — The type of record or row key used in this extended memory table. Valid types are as follows: ALL STRING (alphabetic), ALL NUMERIC (numeric), NUMERIC AND STRING (alphanumeric), ALL BINARY, and BINARY AND STRING.

Key Length — The total length, in bytes, of the primary key to this extended memory table.

Key Algorithm — The algorithm used to hash the key from this file or table in building the optimum hash table. The Extended Memory Table Build utility tries all combinations of allowed hash algorithms, key algorithms, and bucket factors until a table with the fewest collisions is built. Valid key algorithms include the following: SINGLE BYTE XOR, DOUBLE BYTE XOR, TRIPLE BYTE XOR, QAUDRUPLE BYTE XOR, SINGLE BYTE SUMMATION, DOUBLE BYTE SUMMATION, and SINGLE BYTE POLYNOMIAL.

Hash Algorithm — The algorithm used to hash key values from this file or table in building the optimum hash table. The Extended Memory Table Build utility tries all combinations of allowed hash algorithms, key algorithms, and bucket factors until a table with the fewest collisions is built. Valid hash algorithms include the following: MULTIPLICATIVE, MID-SQUARE, and MODULO DIVIDE.

Bucket Factor — The bucket factor used in building the hash table. The bucket factor is the ratio of the number of buckets that will exist as opposed to the actual number of data records in the hash table. The Extended Memory Table Build utility tries all combinations of allowed hash algorithms, key algorithms, and bucket factors until a table with the fewest collisions is built.

Total Buckets — The total number of buckets used for all combinations of allowed hash algorithms, key algorithms, and bucket factors attempted.

Total Buckets Used — The total number of buckets used in the optimum hash table. The Extended Memory Table Build utility tries all combinations of allowed hash algorithms, key algorithms, and bucket factors until a table with the fewest collisions is built.

Percent Buckets Used — The percentage of buckets containing data items in the optimum hash table.

Total Barrels — The total number of barrels used in the optimum hash table. A barrel is equal to a group of buckets.

Total Collisions — The total number of collisions that occurred on the optimum hash table. A collision occurs when two or more data records or rows hash to the same bucket.

Percent Collisions — The percentage of collisions that occurred on the optimum hash table. A collision occurs when two or more data records or rows hash to the same bucket.

Total Collision Chains — The total number of collision chains that occurred on the optimum hash table. A collision chain occurs when two or more data records or rows hash to the same bucket.

Longest Collision Chain — The longest collision chain that occurred on the optimum hash table. A collision chain occurs when two or more data records or rows hash to the same bucket.

Average Collision Chain — The average length of all collision chains that occurred on the optimum hash table. A collision chain occurs when two or more data records or rows hash to the same bucket.

Build Attempts — The total number of combinations attempted to obtain the optimum hash table.

Trailer Page

The trailer page is printed at the end of the Extended Memory Table Build report. It summarizes the total number of swap files successfully built, the total number of swap files that failed, as well as the date and time processing completed.

```
BASE24                EXTENDED MEMORY TABLE BUILD REPORT                PAGE:  n

    EMT Swap Files Built:

                        SCMEMT
                        SSMEMT

    EMT Swap Files Failed:

    End Date:          YY/MM/DD
    End Time:          HH:MM:SS
```

EMT Swap Files Built — The extended memory tables successfully built in this run.

EMT Swap Files Failed — The extended memory tables that failed to be built in this run.

End Date — The date, in the format specified by the DATE-DISPLAY-FORMAT LCONF param, this run of the Extended Memory Table Build utility ended.

End Time — The time of day (HH:MM:SS) this run of the Extended Memory Table Build utility ended.

Section 5

Event Suppression

Event suppression provides a means of reducing the number of times a log message is generated by object-oriented ITS processes. Event suppression may be desirable when multiple processes report the same error condition or when a process reports the same error continually.

Event suppression is a unique feature of the ITS environment. Event suppression is not currently supported by procedural BASE24 applications, although the XPNET process does support its own suppression mechanism. For more information on XPNET process event suppression, refer to the *NET24-XPNET Event Management Manual*.

This section describes how event suppression works and how it is configured to the ITS environment. It includes the following topics:

- An overview of event suppression
- A discussion of event suppression methods
- Configuration requirements for ITS object and Logical Network Configuration File (LCONF) assigns and params
- A discussion of event suppression configuration in the Event Suppression File (ESF)
- A discussion of event suppression performance impacts

Warning: Indiscriminate use of event suppression may adversely affect your ability to monitor system processing by limiting your ability to detect significant processing events if and when they should occur. Therefore, any attempts at event suppression should not be undertaken until after you have had significant experience running your ITS software in a production environment and you are familiar with the types of event messages generated.

Overview

This subsection provides an overview of event suppression processing and introduces each component involved in determining whether or not an event is suppressed.

Note: Events associated with file errors should never be suppressed, because the same event can be generated with different file codes.

Event Object

Event suppression is activated by EMS subsystem IDs (SSIDs) and event numbers. The Event object suppresses events for an ITS system. If an event generated by a process is suppressed at the local process level, the Event object uses the Event Suppression File (ESF) extended memory table to determine whether the event is logged or suppressed. If an event generated by a process is suppressed at the network level, the Event object in the local process reads the Log Permission Process Partition File (LPF) to determine the Log Permission process that handles events generated by the SSID for the event. It then sends a request to the appropriate Log Permission process to determine whether the event is logged or suppressed. The Event object stores data relating to the events it is suppressing and the duration the event is to be suppressed. If a Log Permission process is not configured or not available, the Event object determines whether or not to suppress an event, based on the ESF extended memory table. The Event object resides in every object-oriented ITS process.

Log Permission Object

The Log Permission object determines whether events generated by multiple processes are logged or suppressed at the network level. The Log Permission object receives Event Log requests from Event objects in multiple processes and determines whether or not to log the event. The Log Permission object maintains global knowledge of the events that have been suppressed. A Log Permission object resides in every Log Permission process. If at least one event is suppressed at the network level, then at least one Log Permission process must be configured per logical network to support a range of SSIDs for which event suppression is arbitrated.

Event Suppression Files

The following files are required or associated with event suppression. These files contain data that determines when and how long an event is suppressed.

Event Suppression File

Events that are to be suppressed are defined for the Event object and the Log Permission object in the Event Suppression File (ESF). If an event is not defined in the ESF, it will never be suppressed. The key to the ESF is the subsystem identifier (SSID) of the event generator and the event number. The Event object determines whether an event should be suppressed each time it encounters an SSID and event defined in the ESF. The ESF also contains information about the suppression levels and suppression methods used by the Event object and the Log Permission object. The ESF is described in the *BASE24 ITS Kernel Files Maintenance Manual*.

Event Suppression File Extended Memory Table

The Event Suppression File (ESF) extended memory table is created from the Event Suppression File (ESF) as a hash table. This table is generated by the Extended Memory Table Build utility and is used to improve event suppression performance.

Note: If you decide not to use event suppression at all, then the ESF extended memory table does not need to be built using the Extended Memory Table Build utility.

Log Permission Process Partition File

The Log Permission Process Partition File (LPF) allows log permission requests to be spread across multiple Log Permission processes. This allows you to specify a range of SSIDs and the Log Permission process responsible for determining whether events generated by processes represented in the range of SSIDs are to be logged or suppressed.

The key to an LPF record is the first SSID handled by the specified Log Permission process. The range of SSIDs handled by a Log Permission process is the SSID of the Log Permission process's LPF record to the SSID of the following LPF record. Thus, each Log Permission process handles a contiguous range of SSIDs bounded by the two LPF records.

If any events are suppressed at the network level, at least one LPF entry is required.

Event Suppression Levels

The suppression level indicates the action to be taken when logging an event. Four levels of event suppression are supported: logged, not logged, process-controlled, and Log Permission process-controlled. Events that are logged, not logged, or process-controlled are managed by the Event object while events that are Log Permission process-controlled are controlled by the Log Permission object.

Logged Events

When you select the logged level of suppression for an event, the event is never suppressed. Thus, an event will **always** be logged when this level of event suppression is selected. This is the default level for handling an event. Because this is the default action to be taken for an event, it is not necessary to configure all SSIDs and event numbers. You would use this level if you were changing from Log Permission process-controlled or process-controlled logging and you want to allow logging while retaining the information used with these levels of logging.

The logged level of event suppression is configured using the Event Suppression File (ESF).

Not Logged Events

When you select the not logged level of suppression for an event, the event is always suppressed. Thus, an event will **never be logged** if this level of suppression is used. This level effectively cancels out an event completely.

The not logged level of event suppression is configured using the Event Suppression File (ESF).

Warning: Extreme caution and careful consideration should be made before any event is specified as *not logged*, since this level effectively prohibits the event from ever being logged.

Process-Controlled Events

When you select process-controlled suppression for an event, the event is suppressed by each individual ITS process without the use of a Log Permission process. There is no global coordination of events with this level because process-controlled suppression is implemented within the Event object in the local process. Each ITS process monitors its own event stream. When an event is logged for which process-controlled suppression is used, subsequent occurrences of the event will be suppressed, based on the time interval for the event suppression period or the number of events suppressed. Thus with this level, any event will appear at least once per object-oriented ITS process before it is suppressed.

Process-controlled suppression is the simplest event suppression mechanism because no messages are sent to the Log Permission process. On the other hand, process-controlled suppression causes more events to be logged than Log Permission process-controlled event suppression, which increases overall system resource consumption.

Process-controlled event suppression is configured using the Event Suppression File (ESF).

Log Permission Process-Controlled Events

When you select Log Permission process-controlled suppression for an event, the event is suppressed across multiple ITS processes. Thus, Log Permission process control is analogous to logical network control. With this level, the global coordination of events is performed by the Log Permission process, which monitors events being logged within a logical network. Multiple Log Permission processes can be used. Log Permission process-controlled suppression is more sophisticated than process-controlled suppression because event suppression is controlled at the network level rather than at the process level. However, fewer events are logged with Log Permission process-controlled logging than with process-controlled logging, which reduces overall system resource usage.

Log Permission process-controlled event suppression is configured using the Event Suppression File (ESF) and the Log Permission Process Partition File (LPF).

Event Suppression Methods

Event suppression methods determine how an event is suppressed. Event suppression methods are only used by network (Log Permission process-controlled) and process-controlled events. They are not used for logged and not logged suppression levels. There are two methods of event suppression—suppression by number of events and suppression by time interval. Suppression does not begin until after the first occurrence of the event. One or both of the methods can be selected for an event. If both methods are selected, whichever condition occurs first causes the event to be logged again.

Suppression by Number of Events

Suppression by number of events causes some attempts to log an event to be suppressed. Each attempt to log the event increases a counter by one. When the counter reaches the defined ceiling, the event is logged and the counter is reset. Until the ceiling is reached, all occurrences of the event are suppressed. This method works well for frequently recurring messages.

If process-level suppression is being used, the number of times an event is generated is recorded per process by the Event object.

If network-level suppression is being used, the number of times an event is generated is recorded for all processes in the logical network by the Log Permission process. The Event object must request permission from the Log Permission process each time to log the event, since the Log Permission process must keep track of each log attempt. An interprocess communication (IPC) is incurred each time.

Suppression by Time Interval

Suppression by time interval causes all subsequent occurrences of an event to be suppressed for a specified time interval, after the event is initially logged once.

If process-level suppression is being used, the time duration is recorded per process by the Event object.

If network-level suppression is being used, the time duration is recorded for all processes in the logical network by the Log Permission process. The Log Permission process returns the timestamp of when the next event may be logged to the ITS process. The ITS process records this timestamp and does not request permission from the Log Permission process until after the timestamp has expired,

avoiding the IPCs to the Log Permission process. If both event suppression methods are used, however, the Event object must request permission from the Log Permission process each time to log the event, since the Log Permission process must keep track of each logging attempt. An IPC is incurred each time suppression by number of events is involved.

ITS Process Configuration

This subsection discusses the configuration requirements needed to support event suppression in an object-oriented ITS process and in a Log Permission process.

Object Requirements

The following kernel objects are required in every object-oriented ITS process:

- Application Network Services object
- Class object
- Event object
- Process Control object
- Trace object
- XPNET Interface object

In addition to the kernel objects, any application-specific objects necessary to build the ITS process are required.

In Log Permission processes, the Log Permission object is required in addition to the above kernel objects.

LCONF Requirements

The following LCONF assigns and params are used in any ITS process that supports event suppression. The Event Suppression File extended memory table is required to support any level of event suppression. For detailed descriptions of these assigns and params, refer to the ***BASE24 Logical Network Configuration File Manual***.

Assigns	Params
ESEMT	PROCESS-EVENT-TABLE-SIZE
LPF	MAX-LPP-PROCESSES
LPP- <i>process</i>	LPP-TIME-LIMIT

Event Suppression Configuration

This subsection describes how various fields on Event Suppression File (ESF) screen 1 must be set to support different types of event suppression.

General Configuration Requirements

The following fields on ESF screen 1 must be set properly to support all levels of event suppression. If a particular event is generated and the SSID and event number are not located in the ESF extended memory table, the default action is to log the event.

SSID — This field must be set to the value assigned to the application in the EMCLDDLS file for your system (for example, \$SYSTEM.SCRIBE.EMCLDDLS). The current SSID values for the kernel include the following. For IEP-specific SSIDs, refer to the *BASE24 IEP Configuration Manual*. For core-specific SSIDs, refer to the product-specific transaction processing manual.

SSID	Description
749	Extended Memory Table Build utility
754	Application Network Services object
755	Class object
759	Event object
761	Process Control object
763	Trace object
764	XPNET Interface object
772	Log Permission object

Event Number — This field must be set to the value specified by the application for the event. You can obtain event numbers by viewing the online event message documentation located in the OK13LOGM subvolume or by viewing the event management service (EMS) template files for each object in its source subvolume.

Each file in the OK13LOGM subvolume contains event messages for a specific BASE24 module or object, including the ITS kernel modules and objects. The files are organized according to product module ID (PMID). All kernel PMIDs begin with ISKN, all integrated endpoint PMIDs begin with ISIE, and all core PMIDs begin with ISCO. One PMID exists for each BASE24 module or object. The file names for ITS kernel modules and objects are identified in section 1.

EMS template files reside in individual object source subvolumes and are named according to the object ID followed by the letters PS. These files are also identified in section 1.

Suppressing an Event at the Network Level

To suppress an event at the network level, set the EVENT STATE field on ESF screen 1 to a value of 0. In addition, set the TIME INTERVAL or NUMBER OF EVENTS TO SKIP fields to nonzero values. Set the TIME UNIT field to the value for the appropriate time unit used with the TIME INTERVAL field value.

At least one record must be added to the Log Permission Process Partition File (LPP) with a value in the FIRST SSID field on LPP screen 1 that is equal to or less than the value added in the SSID field on ESF screen 1 for this particular event. An LCONF assign must also exist in the form of LPP-*process*, which contains the PPD name of a configured Log Permission process, where *process* matches the value set in the LPP ASSIGN NAME field on LPP screen 1 and the symbolic process name assigned to the Log Permission process in the XPNET system.

Suppressing an Event at the Process Level

To suppress an event at the process level, set the LOG STATE field on ESF screen 1 to a value of 1. In addition, set the TIME INTERVAL or NUMBER OF EVENTS TO SKIP fields for this ESF record to nonzero values. Set the TIME UNIT field to the value for the appropriate time unit used with the TIME INTERVAL field value.

Never Logging an Event

To configure an ITS process to never log a particular event, set the LOG STATE field on ESF screen 1 to a value of 2. No other field values are required.

Always Logging an Event

To configure an ITS process to always log a particular event, set the LOG STATE field on ESF screen 1 to a value of 3. No other field values are required.

Performance Considerations

The following paragraphs discuss performance-related topics you should consider when configuring event suppression for the ITS environment.

Transaction Path Impacts

The event suppression mechanism has been designed with enough flexibility to allow you to configure event suppression to fit your particular needs. As with many features, there are trade-offs between functionality and cost in terms of additional transaction path length. You should carefully consider all your configuration options to achieve the most desirable configuration for your environment. Event suppression should not be carried out to such an extreme that you can no longer monitor your system effectively, and should be used with careful discretion. Event suppression does not add any measurable increase in transaction path length, except when log permission requests are sent to the Log Permission process. In this case, the cost of an additional IPC is incurred. Therefore, you should restrict the number of events that need to be sent to the Log Permission process to only a few in number, causing only a few requests to incur the cost of an additional IPC. Conversely, you can achieve a noticeable decrease in transaction path cost when certain events are not logged.

Arbitration of Events by Logical Network

Log Permission process-controlled event suppression is slightly more costly than the process-controlled event suppression in terms of transaction path length because of the cost incurred for the IPC sent to the Log Permission process. In addition, specifying a time interval is more efficient than specifying the number of events to skip either alone or in conjunction with the time interval. If only the time interval is specified, the Log Permission process returns the timestamp of when the event can next be logged, thus avoiding any IPCs to the Log Permission process during the time interval.

The purpose of the time interval delay is to provide an additional buffer between the Log Permission process and the processes asking permission, so that the Log Permission process does not become a bottleneck in the ITS system.

Log Permission process-controlled event suppression ensures that the same event issued by multiple processes is not logged multiple times within the specified count and time interval.

Arbitration of Events by Process

Process-controlled event suppression avoids the cost of an additional IPC to the Log Permission process because the Event object itself determines whether the event is logged. With process-controlled event suppression, there is essentially no performance difference between specifying the time interval, number of events to skip, or both. A reasonable time period, however, should be used to ensure that the event is not suppressed for too long a period of time.

Process-controlled event suppression can cause a relatively brief burst of event messages because one event message per object-oriented ITS process is logged at each interval. Unless multiple object-oriented ITS processes of the same type are configured, events from unique process types should be process-controlled because it is more efficient and there is no advantage in going to the Log Permission process.

Not Logging

The not logged level of event suppression is the most efficient option in terms of performance, because the IPC to the Log Permission process is avoided as well as the costs associated with writing the event to the log. The do not log option, however, is also dangerous because anyone watching or analyzing the log will not see the event message when the event occurs. Careful consideration has gone into determining the messages logged by ACI products.

Warning: Extreme caution and careful consideration should be made before any event is specified with a log state of do not log the event in the Event Suppression File (ESF).

Logging

The logged level of event suppression indicates that the event is always logged and is no different than the logging functionality provided without event suppression. Except in test or problem resolution cases, if an event is to be logged, it should not be added to the Event Suppression File (ESF) to keep the size of the Event Suppression File extended memory table to a minimum. The only time the log the event value should be used is for rare cases, when an event that is normally suppressed is to be temporarily logged.

ACI Worldwide Inc.

Glossary

This glossary defines terms related to the configuration of an integrated transaction services (ITS) system.

Actor — An object that performs a service requested by a client object. The actor object always responds to the client's request for service by sending a member function response to the client object.

Class — A set of objects sharing a common structure and behavior. A class acts as a template for an object.

Class can also refer to a grouping of processes or servers that perform similar functions. A server class is a group of processes that perform similar functions. Any process associated with a particular server class can service a transaction sent to that server class. An endpoint class is a group of entities through which messages are sent or received. An endpoint class is physically represented by a group of processes or stations on the network. An endpoint is logically represented by an endpoint class.

Client — An object that requests an actor object to perform a function. The client object always requests services by sending a member function request to the actor object.

Core objects — Objects that provide transaction authorization functionality that is essential to the successful processing of most applications. These objects are referred to as core objects. Typically, a core object resides only in the process where it is used. An example of a core object is any authorization object.

EMT — The Extended Memory Table File. The EMT is used to define shared extended memory tables that are built using the Extended Memory Table Build utility or that can be reallocated by an object using the EMT Warmboot command.

Encapsulation — Refers to the presence of objects—self-contained modules of code defining program data and the operations that can be performed upon that data. An object-oriented (OO) program is built entirely from these self-contained

objects, which communicate by sending messages to one another. In an encapsulated program, the program data and the functions that act upon that data are combined.

Endpoint — An entity through which messages are sent or received, such as a device, interchange, or host. An endpoint is physically represented by a process or station in the network. An endpoint is logically represented by an endpoint class.

Event suppression — A means of reducing the number of times an event message is generated. Event suppression is needed when multiple processes report the same error condition or when the same error is continually reported by the same process.

Extended memory table — Data accessed in processor memory rather than from disk. If the data is obtained from a file or table, the data is maintained on disk as well as in memory. Memory access allows for faster data access than disk access.

Frame — Space in memory that contains local variables, saved registers, and parameters to called procedures. Each object in an ITS process requires its own frame.

Integrated Endpoint process — A satellite process to the XPNET process that is composed from kernel and integrated endpoint objects and is responsible for message routing, maintaining transaction context, and Measure counter manipulation. An Integrated Endpoint process always contains an endpoint-specific Endpoint Class Handler object (EPCH).

Integrated Server process — Any server process running in the ITS environment that is composed from objects. An Integrated Server process is composed from the following kernel objects as well as application-specific objects: Application Network Services, Class, Event, Process Control, Trace, and XPNET Interface. Among other things, an Integrated Server process provides business transaction services, such as: card management, authorization, external interface management, and transaction logging.

Integrated Transaction Services (ITS) — A framework for designing, developing, configuring, and controlling ACI products that utilize an object-oriented design philosophy.

Kernel — A set of objects, files, and process control functions that serves as the base of all ITS applications and provides essential services to an application, such as process and object configuration, event logging, process management, and memory management.

Lexicon — A logical group of messages used to provide communication between objects. For objects to be able to converse with each other they must share at least one of the same lexicons. Each message in a lexicon requests a single member function or responds to a single member function.

Member function — An action that one object can request another object to perform. Because each object is responsible for different data and for the functions related to that data, the member functions supported by each object are different.

Object — A combination of privately held data and procedural statements that act upon that data, called by sending it a message. The object interprets the message and performs the appropriate action. In simple terms, an object is a package of code which performs specific functions. Objects typically encompass specific processing functions that allow for small, easily managed units of code that can accommodate high transaction volumes.

Object ID — An identification number for an object. Object IDs are configured for an object in the Object File (OBJ). The object IDs provided by ACI should not be changed.

Object-oriented design — A software development methodology utilizing small, reusable, easily managed units of code called objects. Objects are used as building blocks to create processes.

Object type — A means of categorizing objects for the purposes of communication, configuration, and control. When an object is called to perform a function, it must be of the correct object type. The object type is used mainly for application objects (that is, core objects).

Procedural — Software designed and implemented with a function-oriented architecture without the use of objects. Non-object oriented software.

Server class — A pool of identical servers whose number can vary with changes in workload, thereby providing application scalability. By assigning an entire group of identical servers to a server class, a client (sending object) does not have to open a specific server of the class; rather, it opens the class and is assigned a link to an available server within that class. A separate queue is typically associated with each server class or process type for the routing of messages.

Subsystem identifier (SSID) — A number identifying an event generator to the Event Management System (EMS). Event suppression is triggered by EMS subsystem IDs (SSIDs) and event numbers.

ACI Worldwide Inc.

Index

A

ACI Performance Monitoring subsystem, [2-14](#)
Actor, definition of, [Glossary-1](#)
ADD PROCESS command, [2-9](#)
Application partitioning, [1-12](#)
Attribute, [1-8](#)

B

Binding requirements, [3-2](#)

C

Class, definition of, [Glossary-1](#)
Client, definition of, [Glossary-1](#)
Configuration files
 customer-configured data, [2-3](#)
 kernel, [1-16](#), [2-3](#)
 preconfigured data, [2-3](#)
 supporting, [1-16](#)
Configuration procedures, [3-3](#)
 adding new assigns, [3-15](#)
 adding new extended memory tables, [3-16](#)
 adding new objects, [3-17](#)
 entering records in the AORF, [3-9](#)
 entering records in the APRF, [3-9](#)
 entering records in the ASGN, [3-4](#)
 entering records in the EMT, [3-4](#)
 entering records in the EORF, [3-9](#)
 entering records in the EPRF, [3-9](#)
 entering records in the LPF, [3-9](#)
 entering records in the OBJ, [3-4](#)
 entering records in the PORF, [3-9](#), [3-11](#)
 entering records in the PRO, [3-4](#), [3-5](#)
 entering records in the PTF, [3-3](#)
 entering records in the SCM, [3-9](#), [3-12](#)
 entering records in the SCT, [3-4](#), [3-7](#)
 entering records in the SPRF, [3-9](#), [3-10](#)
 introduction, [1-17](#)
 modifying preconfigured default data, [3-15](#)
 process binding, [3-2](#)
Core objects
 definition of, [Glossary-1](#)
 introduction, [1-6](#)
Customer-configured data, [2-3](#)

E

EMT WARMBOOT command, [4-4](#)
Encapsulation
 definition of, [Glossary-1](#)
 feature of ITS environment, [1-2](#)
 feature of ITS objects, [1-4](#)
Endpoint, definition of, [Glossary-2](#)
ESF
 see Event Suppression File (ESF)
Event message documentation, [1-19](#)
Event messages, [1-18](#)
Event object, [5-2](#)
Event suppression
 always logging an event, [5-11](#)
 configuration, [5-9](#)
 definition of, [Glossary-2](#)
 files, [5-3](#)
 introduction, [1-15](#), [5-1](#)
 ITS process configuration, [5-8](#)
 LCONF requirements, [5-8](#)
 levels, [5-4](#)
 methods, [5-6](#)
 network level, [5-10](#)
 never logging an event, [5-10](#)
 overview, [5-2](#)
 performance considerations, [5-12](#)
 process level, [5-10](#)
Event Suppression File (ESF)
 event suppression configuration, [5-3](#)
 source file for ESF extended memory table, [4-6](#)
Event Suppression File (ESF) extended memory table
 event suppression usage, [5-3](#)
 source file, [4-6](#)
Event suppression files
 Event Suppression File (ESF), [5-3](#)
 Event Suppression File (ESF) extended memory table, [5-3](#)
 Log Permission Process Partition File (LPF), [5-3](#)
Event suppression levels
 Log Permission process-controlled events, [5-5](#)
 logged events, [5-4](#)
 not logged events, [5-4](#)
 process-controlled events, [5-5](#)
Event suppression methods
 by number of events, [5-6](#)
 by time interval, [5-6](#)
EVENTCX file, [1-18](#)

Extended memory table
 definition of, [Glossary-2](#)
 introduction, [1-9](#)

Extended Memory Table Build Report, [4-10](#)
 cover page, [4-10](#)
 hash table summary page, [4-11](#)
 trailer page, [4-14](#)

Extended Memory Table Build utility
 allocating an extended memory table, [4-4](#)
 illustration of, [4-2](#)
 introduction, [4-1](#)
 LCONF assigns and params, [4-7](#)
 overview, [4-2](#)
 PIN support, [4-8](#)
 processing summary report, [4-10](#)
 screen startup, [4-8](#)
 source files and tables, [4-6](#)
 TACL startup message, [4-8](#)

F

Flat segments, [1-14](#)

Frame, definition of, [1-4](#), [Glossary-2](#)

H

High PIN support
 Extended Memory Table Build utility, [4-8](#)
 overview, [1-13](#)

I

Integrated endpoint objects, [1-7](#)

Integrated Endpoint process, definition of, [Glossary-2](#)

Integrated Server process, definition of, [Glossary-2](#)

Integrated transaction services (ITS)
 definition of, [Glossary-2](#)
 introduction, [1-2](#)

Integrated Transaction Services Control server
 process, [2-8](#)

Integrating the ITS environment
 application partitioning, [1-12](#)
 number of processes, [1-11](#)
 server classes, [1-11](#)
 with procedural applications, [1-11](#)

ITS
 see Integrated transaction services (ITS)

K

Kernel file dependencies, [3-3](#)

Kernel file relationships, [3-3](#)

Kernel objects, [1-6](#)

Kernel, definition of, [Glossary-2](#)

L

Large-memory model, [1-14](#)

LCONF
 see Logical Network Configuration File (LCONF)

LCONF WARMBOOT command, [4-4](#)

Lexicon, definition of, [Glossary-3](#)

Lexicons, [1-7](#)

LIBRARY attribute, [2-12](#)

Log Permission object, [5-2](#)

Log Permission process, [5-2](#)

Log Permission Process Partition File (LPF), [5-3](#)

Logical Network Configuration File (LCONF), [2-7](#)

Logical Network Configuration File (LCONF) assigns
 EMT-SWAPVOL, [4-7](#)
 ESEMT, [4-7](#), [5-8](#)
 ESF, [4-7](#)
 LPF, [5-8](#)
 LPP-*process*, [5-8](#)
 NETLIST-PROG, [4-7](#)
 OBJ, [4-7](#)
 SCM, [4-7](#)
 SCMEMT, [4-7](#)
 SPANCTL, [4-7](#)
 SSMEMT, [4-7](#)

Logical Network Configuration File (LCONF) params
 ALT-TIM-LMT, [4-7](#)
 CONSOLE-PRT, [4-7](#)
 CPFEMT-MAX-TBL-SIZE, [4-7](#)
 CUSTEMT-MAX-TBL-SIZE, [4-7](#)
 DATE-DISPLAY-FORMAT, [4-7](#)
 DESTEMT-MAX-TBL-SIZE, [4-7](#)
 ESEMT-MAX-TBL-SIZE, [4-7](#)
 FPFEMT-MAX-TBL-SIZE, [4-7](#)
 IDFEMT-MAX-TBL-SIZE, [4-7](#)
 IMTEMT-MAX-TBL-SIZE, [4-7](#)
 INTFEMT-MAX-TBL-SIZE, [4-7](#)
 IRTEMT-MAX-TBL-SIZE, [4-7](#)
 KEYAEMT-MAX-TBL-SIZE, [4-7](#)
 LPP-TIME-LIMIT, [5-8](#)
 MAX-LPP-PROCESSES, [5-8](#)
 NETLIST-CPU, [4-7](#)
 NETLIST-PRI, [4-7](#)
 PRI-TIM-LMT, [4-7](#)
 PROCESS-EVENT-TABLE-SIZE, [5-8](#)
 SCMEMT-MAX-TBL-SIZE, [4-7](#)
 SSMEMT-MAX-TBL-SIZE, [4-7](#)
 THFEMT-MAX-TBL-SIZE, [4-7](#)
 TLFEMT-MAX-TBL-SIZE, [4-7](#)

Low PIN support, [4-8](#)

LPF
 see Log Permission Process Partition File (LPF)

M

Measure counter rates configuration, [2-14](#)

Member function
definition of, [Glossary-3](#)
introduction, [1-5](#)

Memory models
large, [1-14](#)
small, [1-14](#)
toggle, [1-14](#)

Memory pool usage, [1-14](#)

N

Network overload management, [2-9](#)

Nucleus process terminology change, [xiv](#)

O

Object
abstraction, [1-4](#)
active, [1-5](#)
configuration of, [1-5](#)
definition of, [Glossary-3](#)
introduction, [1-4](#)
passive, [1-5](#)

Object architecture, [1-3](#)

Object classes
core, [1-6](#)
integrated endpoint, [1-7](#)
introduction, [1-6](#)
kernel, [1-6](#)

Object ID, definition of, [Glossary-3](#)

Object type, definition of, [Glossary-3](#)

Object-oriented design, definition of, [Glossary-3](#)

P

Pathcom, [4-5](#)

PIN

See Process identification number (PIN)

PPD attribute, [2-9](#)

Procedural, definition of, [Glossary-3](#)

Process
object-oriented, [1-3](#)
procedural, [1-3](#)
type, [1-3](#)

Process identification number (PIN), [1-13](#)

Process type, [1-3](#)

Q

QMI attribute, [2-9](#)

QMT attribute, [2-9](#)

S

Server class
definition of, [Glossary-3](#)
external, [1-8](#)
internal, [1-8](#)
introduction, [1-8](#)
type, [1-9](#)

Small-memory model, [1-14](#)

Source class, [2-9](#)

Source Class Map File (SCM), [4-6](#)

Source Class Map File (SCM) extended memory
table, [4-6](#)

Source symbolic map extended memory table, [4-6](#)

SOURCECLASS attribute, [2-9](#)

Subsystem identifier, definition of, [Glossary-3](#)

T

Template files, names of ACI source files, [1-18](#)

X

XPNET process definition, [xiv](#)

ACI Worldwide Inc.